



Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Ingeniería Mecatrónica

Reporte de Trabajo Terminal 1

Control de posición de robot manipulador usando redes neuronales cuantizadas

Autor:

Mendoza Díaz Ramiro

Asesor:

Dr. Jorge Alejandro Juárez Lora

Profesores:

Fernandez Nava Cecilia
Martínez Solis Sandra

Portada con firmas

(Añadir la portada de firmas aquí)

Dedicatoria...

Agradecimientos

Agradezco a...

Índice

Resumen	VI
Abstract	VI
Palabras clave	VII
Índice de figuras	VIII
Índice de tablas	IX
Nomenclatura	X
Simbología	XII
Introducción	1
0.0.1. Multiplicación de matrices, procesamiento de señales y problemas de computación en tareas robóticas	2
0.0.2. Cuello de botella de Von Neumann	2
0.0.3. Proyección a futuro de la problemática energética	3
0.0.4. Objetivo general	4
0.0.5. Objetivos específicos TT1	4
0.0.6. Objetivos específicos TT2	5
0.0.7. Técnicas de control clásico para robots manipuladores	6
0.0.8. Transición hacia el control basado en Inteligencia Artificial (IA)	7
0.0.9. Estado del arte	8
1. Marco de Referencia	9
1.1. Marco teórico	9
1.1.1. Modelado y Control de Robots Manipuladores	9
1.1.2. Redes Neuronales Artificiales para el Control Robótico	12
1.1.3. Estrategias de Optimización de Redes Neuronales	12
1.2. Marco procedimental	17
1.2.1. Metodología mecatrónica	17
2. Diseño del sistema	18
2.1. Diseño conceptual	18
2.1.1. Necesidades - requerimientos	19
2.1.2. Arquitectura funcional (funciones)	19
2.1.3. Arquitectura física (módulos)	22
2.1.4. Propuestas solución	23
2.1.5. Validación	24
2.1.6. Selección diseño conceptual	26
2.2. Diseño detallado	27
2.2.1. Detalle módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real	27

2.2.2.	Detalle módulo 2 (M_2) – Control Dinámico Analítico en Tiempo Real	29
2.2.3.	Detalle módulo 3 (M_3) – Inteligencia Artificial y Motor Cuantizado	31
2.2.4.	Detalle módulo 4 (M_4) – Supervisión, Telemetría y Adquisición	33
2.2.5.	Integración del sistema mecatrónico	34
3.	Implementación del sistema	36
3.1.	Implementación módulo 1 (M_1): Planta robótica y entorno Sim-to-Real	36
3.1.1.	Verificación (M_1)	37
3.2.	Implementación módulo 2 (M_2): Control dinámico analítico en tiempo real	38
3.2.1.	Verificación (M_2)	40
3.3.	Implementación módulo 3 (M_3): Inteligencia artificial y motor cuantizado	41
3.3.1.	Verificación (M_3)	45
3.4.	Implementación módulo 4 (M_4): Supervisión, telemetría y adquisición .	47
3.4.1.	Verificación (M_4)	48
3.5.	Integración del sistema mecatrónico	49
3.5.1.	Verificación del sistema mecatrónico	49
4.	Análisis de resultados	50
4.1.	Análisis de ingeniería	50
4.2.	Análisis de costos	50
4.3.	Análisis de valor	50
	Conclusiones	52
	Recomendaciones y trabajo a futuro	52
	Apéndices	56
	Anexos	56
4.3.1.	Recursos Estimados e Infraestructura	56
4.3.2.	Recursos Humanos	56
4.3.3.	Recursos Materiales y Económicos	56
4.3.4.	Infraestructura Requerida	58
4.3.5.	Planeación de Actividades (Cronograma General)	59
	Fuentes – Referencias	61
	Glosario	65

Resumen

La robótica moderna demanda sistemas de control cada vez más complejos para que los manipuladores ejecuten tareas aún más desarrolladas con precisión y adaptabilidad. Si bien la inteligencia artificial, y en particular las redes neuronales profundas (DNNs), ofrecen soluciones potentes, su elevado costo computacional y energético restringe su aplicación en plataformas robóticas con cómputo al filo. Este proyecto de investigación se centra en este desafío, motivado por la necesidad crítica de desarrollar controladores robóticos energéticamente eficientes y de validar cuantitativamente sus beneficios.

Se plantea el problema de cómo implementar un control de posición para un robot manipulador que sea preciso y, fundamentalmente, de bajo consumo, mediante el uso de redes neuronales cuantizadas (QNNs) desplegadas en hardware reconfigurable (FPGA). El objetivo general de este trabajo es diseñar, implementar y evaluar comparativamente el desempeño energético, la utilización de recursos de hardware y la precisión del control de un sistema basado en QNNs, contrastándolo con un control PID clásico y una red neuronal de punto flotante (FP12-NN), todos operando sobre la misma plataforma FPGA.

La propuesta de solución abarca el desarrollo de estos tres enfoques de control, la aplicación de técnicas de cuantización para generar modelos QNN con distintos niveles de precisión, y la ejecución de un plan experimental para medir y analizar las métricas de rendimiento y consumo energético. Se espera que los resultados demuestren la viabilidad y las ventajas energéticas de las QNNs, aportando una metodología y datos concretos para el diseño de futuros sistemas de control robótico energéticamente sostenibles.

Abstract

Modern robotics demands increasingly complex control systems for manipulators to perform more advanced tasks with precision and adaptability. While artificial intelligence—particularly deep neural networks (DNNs)—offers powerful solutions, their high computational and energy costs limit their use in robotic platforms with edge computing. This research project addresses this challenge, driven by the critical need to develop energy-efficient robotic controllers and to quantitatively validate their benefits.

The central problem is how to implement a position control system for a robotic manipulator that is both accurate and, fundamentally, low-power, using quantized neural networks (QNNs) deployed on reconfigurable hardware (FPGAs). The main objective of this work is to design, implement, and comparatively evaluate the energy performance, hardware resource usage, and control accuracy of a QNN-based system, benchmarking it against a classical PID controller and a floating-point neural network (FP12-NN), all running on the same FPGA platform.

The proposed solution involves developing all three control strategies, applying quantization techniques to produce QNN models with varying precision levels, and executing an experimental plan to measure and analyze performance and energy consumption metrics. The expected outcome is to demonstrate the feasibility and energy advantages of QNNs, providing both a methodology and concrete data for the design of future energy-efficient robotic control systems.

Palabras clave

Redes Neuronales Cuantizadas, Control de Robots, FPGA, Eficiencia Energética, Robótica, Computo al filo.

Índice de figuras

1.	Arquitectura Von Neumann original [1]	2
2.	Reducción de bits de entrada y de precisión en la salida por la cuantización [2]	4
3.	Diagrama simplificado de un control PID [3].	6
4.	Estructura de una red neuronal (ANN) [4].	7
5.	Comparación de una ANN y una DNN [5].	8
6.	Diagrama de la arquitectura funcional y flujo de datos en tiempo real del sistema de control inteligente bajo la metodología ágil.	21
7.	Diagrama de integración mecatrónica global: interconexión de nodos, buses en tiempo real y acoplamiento temporal de la arquitectura híbrida.	34

Índice de cuadros

1.	Acrónimos y términos técnicos empleados en el proyecto.	X
2.	Simbología matemática y magnitudes físicas del sistema mecatrónico. .	XII
3.	Requerimientos funcionales y no funcionales del sistema de control mecatrónico.	20
4.	Matriz de selección conceptual de Pugh para las alternativas del sistema de control.	26
5.	Parámetros de Denavit-Hartenberg modificados del robot UR3e.	28
6.	Topología de la red neuronal compensadora de dinámica (UR3eNetwork). .	31
7.	Comparativa de recursos entre el modelo original en FP32 y el modelo cuantizado en INT8.	32
8.	Presupuesto temporal consolidado del lazo crítico de control mecatrónico a 500 Hz ($T_s = 2000 \mu s$).	35
9.	Métricas experimentales de verificación para el módulo M_1 (Planta Sim-to-Real).	38
10.	Métricas experimentales de verificación para el módulo M_2 (Control Dinámico Clásico).	41
11.	Resultados experimentales de verificación: Red FP32 vs. Red Cuantizada INT8 (M_3).	47
12.	Métricas experimentales de verificación para el módulo M_4 (Telemetría y Adquisición).	49
13.	Matriz comparativa de verificación del sistema mecatrónico frente a los requerimientos de diseño.	51

Nomenclatura

En la Tabla 1 se presentan los acrónimos, términos técnicos y abreviaturas especializadas utilizadas a lo largo de este documento, ordenadas alfabéticamente.

Cuadro 1: Acrónimos y términos técnicos empleados en el proyecto.

Término / Sigla	Significado y Descripción Técnica
AI / IA	Inteligencia Artificial (<i>Artificial Intelligence</i>).
ANN	Red Neuronal Artificial (<i>Artificial Neural Network</i>).
CIM	Computación en Memoria (<i>Computing-In-Memory</i>).
CoM	Centro de Masa (<i>Center of Mass</i>) de un eslabón rígido.
CTC	Control por Par Computado (<i>Computed Torque Control</i>).
DH	Denavit-Hartenberg (convención cinemática modificada de parámetros geométricos).
DNN	Red Neuronal Profunda (<i>Deep Neural Network</i>).
FP32	Formato de punto flotante de precisión simple de 32 bits (estándar IEEE 754).
FPGA	Matriz de Puertas Lógicas Programable en Campo (<i>Field-Programmable Gate Array</i>).
GDL	Grados de Libertad (<i>Degrees of Freedom</i> , DoF).
INT8	Formato numérico entero con signo de 8 bits (rango $[-128, 127]$).
MAC	Operación de Multiplicación y Acumulación (<i>Multiply-Accumulate</i>).
MAE	Error Absoluto Medio (<i>Mean Absolute Error</i>).
MCAP	Formato de almacenamiento eficiente para grabación de datos y telemetría en ROS 2.
ML	Aprendizaje Automático (<i>Machine Learning</i>).
MLP	Perceptrón Multicapa (<i>Multilayer Perceptron</i>).
MSE	Error Cuadrático Medio (<i>Mean Squared Error</i>).
ONNX	Formato Abierto para Intercambio de Redes Neuronales (<i>Open Neural Network Exchange</i>).
PD / PID	Controlador Proporcional-Derivativo / Proporcional-Integral-Derivativo.
PINN	Red Neuronal Informada por la Física (<i>Physics-Informed Neural Network</i>).
PTQ	Cuantización Post-Entrenamiento (<i>Post-Training Quantization</i>).
QAT	Entrenamiento Consciente de Cuantización (<i>Quantization-Aware Training</i>).
QNN	Red Neuronal Cuantizada (<i>Quantized Neural Network</i>).

Continúa en la siguiente página...

Término / Sigla	Significado y Descripción Técnica
QoS	Calidad de Servicio (<i>Quality of Service</i>) en protocolos de transporte ROS 2.
RNEA	Algoritmo Recursivo de Newton-Euler (<i>Recursive Newton-Euler Algorithm</i>).
RMSE	Raíz del Error Cuadrático Medio (<i>Root Mean Square Error</i>).
ROS / ROS 2	Sistema Operativo de Robots (<i>Robot Operating System</i> versión 2).
RT	Tiempo Real (<i>Real-Time</i>).
SNN	Red Neuronal de Impulsos (<i>Spiking Neural Network</i>).
UR3e	Robot manipulador colaborativo industrial de 6 GDL de Universal Robots.
URDF	Formato Unificado de Descripción de Robots (<i>Unified Robot Description Format</i>).
VoBo	Visto Bueno de aprobación emitido por el jurado o asesores.

Simbología

En la Tabla 2 se detallan las variables matemáticas, operadores, vectores y matrices empleados en la formulación cinemática, dinámica y de control, especificando sus unidades en el Sistema Internacional (SI).

Cuadro 2: Simbología matemática y magnitudes físicas del sistema mecatrónico.

Símbolo	Descripción Física o Matemática	Unidad (SI)
<i>Cinemática y Variables Articulares:</i>		
$\mathbf{q} = [q_1, \dots, q_6]^T$	Vector de posiciones articulares generalizadas	rad
$\dot{\mathbf{q}} = [\dot{q}_1, \dots, \dot{q}_6]^T$	Vector de velocidades articulares	rad/s
$\ddot{\mathbf{q}} = [\ddot{q}_1, \dots, \ddot{q}_6]^T$	Vector de aceleraciones articulares	rad/s ²
$\mathbf{q}_d, \dot{\mathbf{q}}_d, \ddot{\mathbf{q}}_d$	Trayectorias articular deseada (posición, velocidad y aceleración)	rad, rad/s, rad/s ²
$\mathbf{e} = \mathbf{q}_d - \mathbf{q}$	Vector de error de seguimiento de posición articular	rad
$\dot{\mathbf{e}} = \dot{\mathbf{q}}_d - \dot{\mathbf{q}}$	Vector de error de velocidad articular	rad/s
α_{i-1}	Parámetro DH modificado: ángulo de torsión de eslabón	rad
a_{i-1}	Parámetro DH modificado: longitud de eslabón	m
d_i	Parámetro DH modificado: desplazamiento en el eje articular	m
θ_i	Parámetro DH modificado: variable angular articular	rad
$\mathbf{R}_i^{i-1}$	Matriz de rotación del marco coordenado i respecto al marco $i - 1$	Adimensional
$\mathbf{P}_i^{i-1}$	Vector de traslación del origen $i - 1$ al origen i expresado en el marco i	m
$\mathbf{z}_0$	Vector unitario sobre el eje de rotación articular $[0, 0, 1]^T$	Adimensional
<i>Cinemática Diferencial y Dinámica RNEA (Paso hacia Adelante):</i>		
$\boldsymbol{\omega}_i$	Vector de velocidad angular del eslabón i expresado en su propio marco	rad/s
$\dot{\boldsymbol{\omega}}_i$	Vector de aceleración angular del eslabón i	rad/s ²
$\dot{\mathbf{v}}_i$	Vector de aceleración lineal del origen del marco coordenado i	m/s ²
$\dot{\mathbf{v}}_{c,i}$	Vector de aceleración lineal del centro de masa del eslabón i	m/s ²
$\mathbf{r}_{c,i}$	Vector de posición local del centro de masa del eslabón i	m
$\mathbf{g} = [0, 0, 9,81]^T$	Vector de aceleración de la gravedad terrestre	m/s ²

Continúa en la siguiente página...

Símbolo	Descripción Física o Matemática	Unidad (SI)
<i>Dinámica Analítica y Esfuerzos RNEA (Paso hacia Atrás):</i>		
m_i	Masa total del eslabón i	kg
I_i	Tensor de inercia baricéntrico del eslabón i expresado en su marco local	kg · m ²
F_i	Fuerza inercial total que actúa sobre el eslabón i	N
N_i	Momento inercial total que actúa sobre el eslabón i	N · m
f_i	Fuerza ejercida por el eslabón $i - 1$ sobre el eslabón i	N
n_i	Par/momento ejercido por el eslabón $i - 1$ sobre el eslabón i	N · m
$\tau = [\tau_1, \dots, \tau_6]^T$	Vector de pares motores netos generalizados aplicados a las articulaciones	N · m
τ_{rnea}	Par dinámico de prealimentación calculado por RNEA	N · m
τ_{cmd}	Par de comando total enviado a las interfaces de esfuerzo de los motores	N · m
$M(q)$	Matriz de masa e inercia del manipulador (6×6)	kg · m ²
$C(q, \dot{q})$	Matriz de términos centrífugos y de Coriolis (6×6)	N · m · s/rad
$g(q)$	Vector de pares gravitacionales articulares (6×1)	N · m
<i>Control Automático y Parámetros de Redes Neuronales:</i>		
K_p, K_d, K_i	Matrices de ganancias de retroalimentación proporcional, derivativa e integral	N · m/rad, N · m/s/rad, N · m/(rad · s)
a_{cmd}	Vector de aceleración de control auxiliar desacoplada	rad/s ²
$X \in \mathbb{R}^{42}$	Vector de entrada a la red neuronal (7 subgrupos de 6 articulaciones)	Dimensiones mixtas
$y \in \mathbb{R}^6$	Vector de salida predicho por la red (par estimado para 6 articulaciones)	N · m
μ_x, σ_x	Vectores de media y desviación estándar para normalización de entrada	Dimensiones mixtas
μ_y, σ_y	Vectores de media y desviación estándar para desnormalización de salida	N · m
S	Factor de escala de cuantización (<i>Scale factor</i>)	Adimensional
Z	Punto cero entero de compensación de cuantización (<i>Zero-point</i>)	Adimensional
q_{int8}	Valor entero cuantizado a 8 bits de un operando o peso	Adimensional
T_s	Periodo de muestreo del bucle de control de tiempo real (10 ms = 100 Hz)	s

Introducción

Los robots manipuladores son componentes esenciales en la automatización industrial, la cirugía asistida, la exploración espacial y una creciente variedad de aplicaciones de servicio, gracias a su capacidad para realizar tareas de manipulación con precisión y repetibilidad [6]. El control efectivo de estos sistemas es fundamental para su desempeño. Tradicionalmente, se han empleado técnicas de control clásico, como el control Proporcional-Integral-Derivativo (PID) o el control por par calculado, que han demostrado ser robustas para tareas definidas y con modelos dinámicos precisos del robot [6]. Sin embargo, la creciente demanda de mayor autonomía, adaptabilidad a entornos no estructurados y la capacidad para manejar interacciones complejas ha impulsado la adopción de enfoques basados en Inteligencia Artificial (IA), particularmente las Redes Neuronales Artificiales (ANNs) y las Redes Neuronales Profundas (DNNs) [7, 8].

A pesar de sus ventajas, la implementación de DNNs en sistemas robóticos conlleva desafíos significativos. Estos modelos suelen ser computacionalmente intensivos, caracterizados por un gran número de parámetros y una alta densidad de operaciones aritméticas, lo que se traduce en un considerable consumo energético y una fuerte demanda de recursos de memoria y procesamiento [9, 10]. Estas exigencias son particularmente problemáticas para las plataformas robóticas autónomas, que operan con fuentes de alimentación y recursos computacionales limitados [11, 12].

El interés de este estudio surge de la necesidad de reducir estos costes computacionales y energéticos. Las Redes Neuronales Cuantizadas (QNNs), que reducen la precisión numérica de los pesos y activaciones de una red, ofrecen una vía prometedora para comprimir modelos y acelerar la inferencia, con el potencial de disminuir drásticamente el consumo energético y el uso de memoria [10, 13].

Este trabajo busca aportar conocimiento práctico y cuantitativo sobre la viabilidad y los beneficios de utilizar QNNs para el control eficiente de robots manipuladores, ofreciendo una perspectiva con un enfoque en la eficiencia energética de diferentes métodos de control de sistemas robóticos.

Enfoque mecatrónico

En este apartado se describe cómo el proyecto evalúa un sistema completo que integra la mecánica del robot, la electrónica de medición y actuación, y el algoritmo de control implementado.

Definición del problema

El control de robots es esencial para su implementación en la automatización industrial o en robótica de consumo, a medida que los robots se emplean en tareas más avanzadas o con entornos cambiantes o dinámicos, los métodos de control se vuelven más complejos y con esto, su coste computacional y energético aumentan [7, 8].

0.0.1. Multiplicación de matrices, procesamiento de señales y problemas de computación en tareas robóticas

Las multiplicaciones de matrices tienen consecuencia directa en la robótica ya que las operaciones matriciales están implicadas en todos sus aspectos, particularmente la multiplicación vector-matriz y las operaciones de multiplicación-acumulación. Las ecuaciones de la cinemática directa e inversa, el modelo dinámico del robot dado por las ecuaciones de Euler-Lagrange o Newton-Euler, así como los diferentes métodos de control pudiendo ir desde un PID con compensaciones dinámicas, hasta los basados en aprendizaje automático y redes neuronales, todos dependen del uso y operación de matrices y vectores.[9, 6]

Este tipo de operaciones se realizan repetidamente a una alta frecuencia, lo que conlleva un costo computacional significativo. La ineficiencia en la realización de estas operaciones provoca un deterioro en el rendimiento en tiempo real, y un incremento en el consumo energético, que tiene su origen en la arquitectura de los equipos de computo actuales.

0.0.2. Cuello de botella de Von Neumann

La arquitectura Von Neumann, es un tipo específico de arquitectura computacional en la que tanto los datos como las instrucciones se almacenan en una memoria común como se muestra en la figura 1 donde se detalla el uso de memoria para los datos e instrucciones.

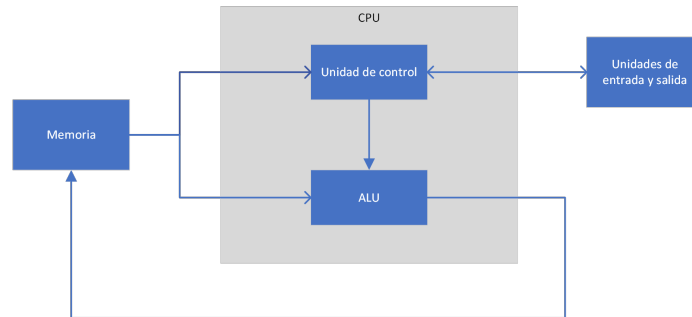


Figura 1: Arquitectura Von Neumann original [1]

Aunque este modelo ha sido fundamental para la computación digital durante muchos años, tiene limitaciones inherentes a su diseño que surgen del uso intensivo de datos, debido a la separación física entre el CPU y la memoria. Esto conlleva la necesidad de transferir datos e instrucciones a través de un bus de memoria compartido, lo que se conoce como el *cuello de botella de Von Neumann*. [1, 14]

Este cuello de botella provoca una latencia significativamente elevada, producto de la espera constante de la CPU por datos e instrucciones desde la memoria, y en un consumo energético considerable debido al intenso tráfico de datos -conocido como *memory wall*. En algoritmos con alta demanda de acceso a memoria y operaciones repetitivas sobre datos (ejemplo, multiplicación de matrices), como un control robótico,

procesamiento en tiempo real y aprendizaje automático, estas limitaciones impactan directamente en el rendimiento y la eficiencia energética del sistema.

0.0.3. Proyección a futuro de la problemática energética

Tanto la robótica como la inteligencia artificial se perfilan hacia su integración en sistemas autónomos capaces de percibir diferentes entornos, aprender de su propia experiencia y tomar decisiones en tiempo real. Debido a la complejidad de estas implementaciones, la demanda computacional ha incrementado trayendo consigo un consumo energético cada vez mayor[12, 15]. Esto presenta un problema para robots autónomos, drones, exoesqueletos de asistencia o sistemas colaborativos humano-robot donde la fuente de energía está suministrada por baterías. Este alto consumo proviene de los sistemas de control que hacen un uso intensivo de datos en arquitecturas de cómputo basadas principalmente en la arquitectura Von Neumann poniendo en juego la viabilidad de dichos sistemas autónomos.

Conforme los modelos de IA se vuelven más profundos y los sensores generan mayores volúmenes de datos, este “muro energético” se vuelve cada vez más pronunciado. Con esta proyección, se resalta la necesidad de la investigación y exploración de diferentes paradigmas computacionales que ofrezcan una mejora significativa a la eficiencia energética.

Justificación

Debido a las restricciones de cómputo actuales en chips tradicionales, surge la necesidad de mejorar su desempeño energético explorando alternativas de control enfocadas en la eficiencia computacional y energética. Este proyecto se centra en la evaluación de un método de control alternativo denominado redes neuronales cuantizadas o Quantized Neural Networks (QNNs) como una vía para lograr una mejora sustancial en la eficiencia energética de los sistemas de control de robots manipuladores. El objetivo principal es realizar una comparación directa del consumo energético de un sistema de control basado en QNNs con un control clásico bajo una plataforma de pruebas como lo puede ser una FPGA. Las redes neuronales cuantizadas reducen sus bits de entrada provocando una reducción en su requerimiento de cómputo y provoca una reducción también en la precisión de salida de dicha red como se ve en la figura 2[16, 17, 18].

El enfoque mecatrónico es esencial, ya que el proyecto evaluará un sistema completo que integra la mecánica del robot, la electrónica de medición y actuación, y el algoritmo de control (QNN o tradicional) implementado en una plataforma de *hardware* específica. La comparación energética se realizará considerando el sistema en su conjunto, ofreciendo una perspectiva realista del impacto de la optimización neuronal en el consumo global del robot durante tareas de control.

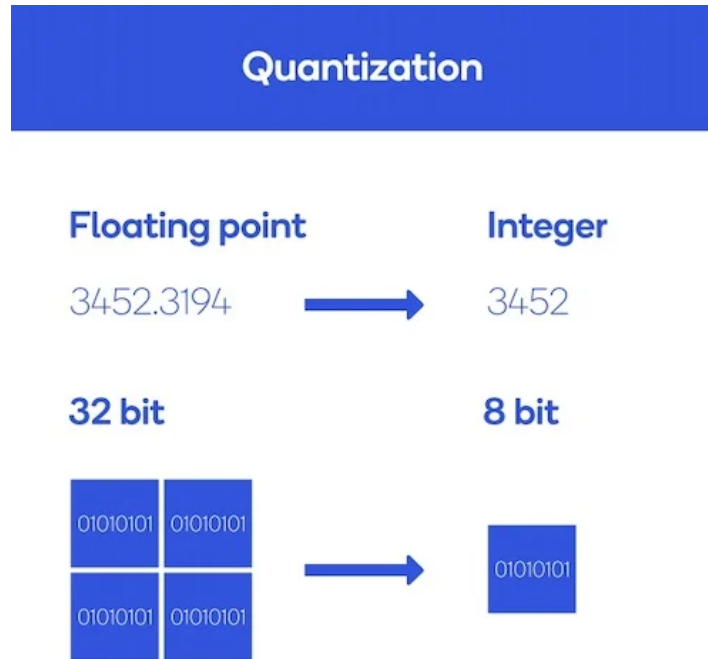


Figura 2: Reducción de bits de entrada y de precisión en la salida por la cuantización [2]

Objetivo

0.0.4. Objetivo general

Evaluar el desempeño energético, de precisión y de recursos utilizados en la implementación de un control de lazo cerrado con una planta robótica con control tradicional y con una red neuronal cuantizada.

0.0.5. Objetivos específicos TT1

- Investigar el estado del arte de las redes neuronales cuantizadas (QNNs) aplicadas al control de sistemas robóticos, las metodologías de cuantización (Post-Training Quantization - PTQ y Quantization-Aware Training - QAT), y las técnicas para la estimación y comparación del consumo energético en plataformas de cómputo embebidas y tradicionales.
- Definir la arquitectura de una red neuronal artificial (ANN) adecuada para el control de posición de un robot manipulador, incluyendo el preprocesamiento de las señales de entrada necesarias para la red.
- Diseñar el sistema de control de posición basado en la ANN seleccionada (sin cuantizar) y establecer las métricas de rendimiento de control que servirán como línea base.
- Establecer una metodología detallada para la cuantización del modelo de red neuronal diseñado, seleccionando al menos dos niveles de precisión distintos (ej.,

INT6 y INT8) y las herramientas de software para llevar a cabo el proceso.

- Diseñar el plan experimental para la comparación energética y de rendimiento, definiendo los escenarios de prueba, las variables a medir (ej., latencia de inferencia, uso de memoria, consumo energético estimado/medido, precisión del control) y la configuración del entorno de pruebas (simulado y/o físico).
- Caracterizar y poner en marcha la plataforma robótica y su interfaz de control, asegurando la capacidad de implementar y evaluar los diferentes esquemas de control.
- Escribir en el documento escrito dichos avances en la investigación.

0.0.6. Objetivos específicos TT2

- Implementar el lazo de control numérico tradicional (ej. PID) de posición del efector final, tanto en simulación numérica, como en físico, con la finalidad de tener un *benchmark* para comparar con la red neuronal quantizada, tanto en desempeño, recurso energético, etc.
- Entrenar, implementar y validar el sistema de control de posición basado en la red neuronal no cuantizada (ANN de punto flotante) en la plataforma robótica, evaluando su rendimiento de control y su perfil de consumo energético como segundo punto de referencia.
- Implementar los sistemas de control de posición basados en las redes neuronales cuantizadas (QNNs) para los niveles de precisión seleccionados en TT1, integrándolos con la plataforma robótica.
- Realizar estudio comparativo de ambas soluciones y evaluar el desempeño energético, de control, de recursos computacionales, y de recursos de hardware implementados.
- Analizar e interpretar los resultados experimentales para cuantificar las ventajas y desventajas de cada enfoque de control, con un énfasis particular en la eficiencia energética lograda por las QNNs y su relación con la precisión del control.
- Documentar la metodología, los resultados obtenidos, las conclusiones del estudio comparativo y las recomendaciones para futuras implementaciones de QNNs en control robótico.

Antecedentes

El control de robots manipuladores es un campo bien establecido con una amplia historia de técnicas desarrolladas para lograr un movimiento preciso y fiable. Antes de la prominencia de los enfoques basados en inteligencia artificial, el control robótico se fundamentaba en gran medida en modelos matemáticos del robot y su entorno.

0.0.7. Técnicas de control clásico para robots manipuladores

Los controladores para robots manipuladores se pueden clasificar ampliamente según el espacio en el que se ejerce el control (espacio articular o espacio de la tarea) y el grado en que utilizan un modelo dinámico del robot.

Una de las estrategias más directas y comúnmente implementadas es el control PID (Proporcional-Integral-Derivativo) independiente para cada articulación. En este enfoque, cada articulación del robot es tratada como un sistema de un solo grado de libertad, y se diseña un controlador PID para regular su posición y velocidad basándose en el error entre el estado deseado y el medido (Lynch & Park, 2017, Capítulo 11.2)[6]. La simplicidad de diseño como se muestra en la figura 3 y sintonización (aunque esta última puede ser tediosa para sistemas multigrado de libertad altamente acoplados) ha hecho del control PID una opción popular, especialmente cuando no se dispone de un modelo dinámico preciso del robot. Para mejorar el rendimiento, a menudo se complementa con un control por prealimentación (*feedforward*), que utiliza el modelo cinemático y, a veces, un modelo dinámico simplificado para calcular los pares/fuerzas articulares nominales necesarios para seguir una trayectoria deseada, ayudando al lazo de realimentación PID a compensar solo las perturbaciones y errores de modelado (Lynch & Park, 2017, Capítulo 11.3)[6].

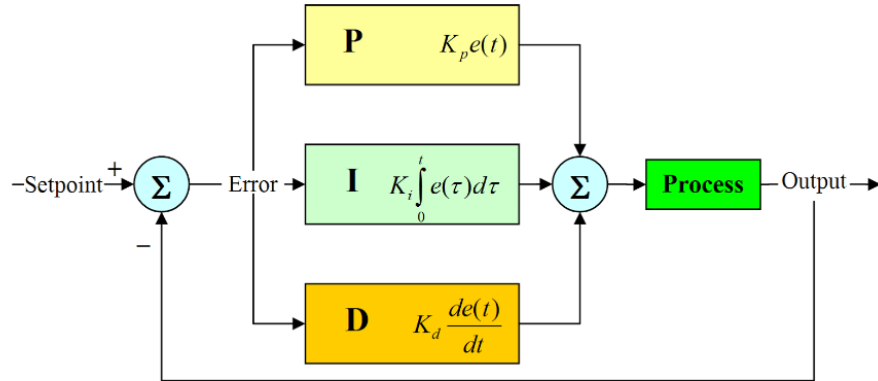


Figura 3: Diagrama simplificado de un control PID [3].

Cuando se dispone de un modelo dinámico mas completo del robot como en la ecuación 1.

$$M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta) = \tau \quad (1)$$

Se pueden implementar estrategias de control basadas en modelo más sofisticadas. Una técnica fundamental es el control por par calculado (*computed torque control*) o control por dinámica inversa (*inverse dynamics control*). Este enfoque busca linealizar y desacoplar la dinámica del robot mediante una ley de control que cancela los términos no lineales y de acoplamiento. La idea es calcular los pares de motor necesarios utilizando

el modelo dinámico inverso y luego aplicar un control lineal simple (como un PID) sobre el sistema linealizado resultante. Bajo la suposición de un modelo dinámico perfecto, el robot se comportaría como un sistema lineal desacoplado, lo que permitiría un control de alta precisión[6].

Otras técnicas de control avanzado incluyen el control adaptativo, que ajusta los parámetros del controlador en línea para compensar incertidumbres en el modelo dinámico del robot o variaciones en la carga útil[19], y el control robusto, diseñado para mantener la estabilidad y el rendimiento a pesar de las incertidumbres acotadas del modelo y las perturbaciones externas. El control en el espacio de la tarea (*task space control*) también es crucial, donde el control se ejerce directamente sobre la posición y orientación del efector final del robot, utilizando el jacobiano para relacionar las velocidades del efector final con las velocidades articulares.

0.0.8. Transición hacia el control basado en Inteligencia Artificial (IA)

Si bien las técnicas de control clásico mencionadas son efectivas en muchas situaciones, su rendimiento puede degradarse significativamente en presencia de dinámicas no modeladas complejas, fricción no lineal, incertidumbres paramétricas grandes o cuando se requiere que el robot opere en entornos altamente no estructurados o interactúe de forma compleja con el entorno. La obtención de modelos dinámicos precisos puede ser costosa y difícil, y la sintonización de controladores PID para sistemas multivariables fuertemente acoplados es a menudo un desafío. Estas limitaciones han motivado la exploración de enfoques basados en inteligencia artificial (IA), y en particular las redes neuronales artificiales (ANNs) de las cuales su estructura se puede apreciar en la figura 4.

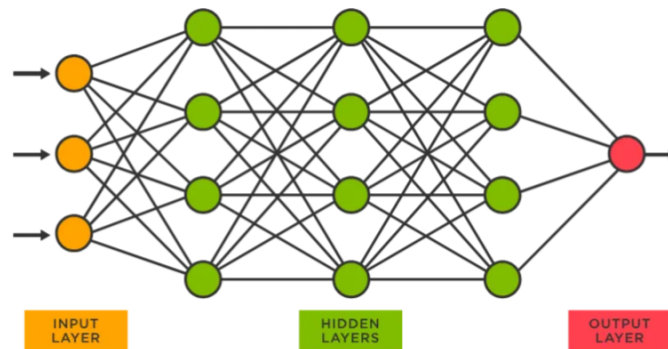


Figura 4: Estructura de una red neuronal (ANN) [4].

Las redes neuronales profundas (DNNs) mostradas en la figura 5, han emergido como herramientas poderosas para el aprendizaje de tareas de control complejas, como la cinemática inversa, el seguimiento de trayectorias y el aprendizaje por refuerzo para la manipulación[8, 20]. Estas técnicas permiten a los robots aprender de la experiencia y adaptarse a variaciones, superando en muchos casos las limitaciones de los controladores tradicionales.

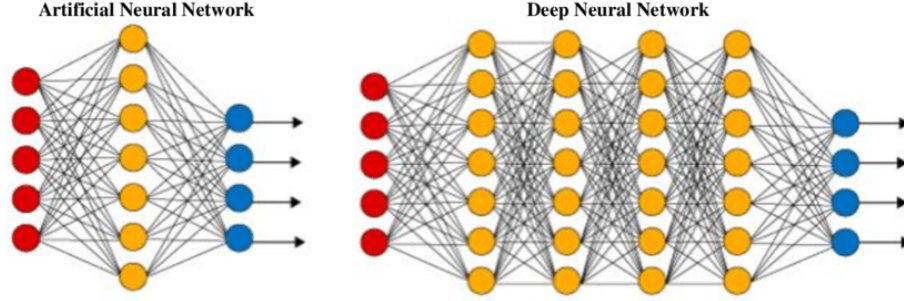


Figura 5: Comparación de una ANN y una DNN [5].

Sin embargo, el éxito y la creciente complejidad de los modelos de DNNs vienen acompañados de una demanda computacional y un consumo energético considerables[9, 12]. El gran número de parámetros y las intensivas operaciones de multiplicación y acumulación (MAC) inherentes a estas redes plantean un desafío significativo para su despliegue en plataformas robóticas, las cuales a menudo operan con recursos de hardware limitados (memoria, capacidad de procesamiento) y restricciones energéticas, especialmente en aplicaciones móviles o autónomas[11, 21]. Esta tensión entre la capacidad de los modelos de IA y las limitaciones de las plataformas de despliegue ha impulsado una vasta investigación en la optimización y compresión de redes neuronales.

0.0.9. Estado del arte

Gestionar de manera eficaz sistemas robóticos en contextos donde la energía es limitada sigue siendo un reto constante y cada vez más relevante. Aunque las arquitecturas de cómputo tradicionales, en su mayoría basadas en el esquema de Von Neumann, han facilitado importantes desarrollos, presentan limitaciones estructurales, especialmente en lo referente al acceso a la memoria y al consumo energético, cuando se utilizan en tareas de control avanzadas o en aplicaciones de inteligencia artificial. Investigaciones recientes han mostrado un interés creciente por mejorar la eficiencia de los modelos de inteligencia artificial, lo cual se refleja en iniciativas como la compresión de redes neuronales y el estudio de enfoques de cómputo alternativos a los tradicionales.

Dentro de este panorama, el cómputo neuromórfico se perfila como una opción innovadora, inspirada en la forma en que el cerebro humano procesa información de manera eficiente y paralela[22]. Este enfoque se basa en la implementación de las redes neuronales de impulsos (SNN), las cuales funcionan a través de eventos discretos (conocidos como *spikes*) y presentan una dinámica temporal inherente, lo que las hace especialmente adecuadas para aplicaciones de control en tiempo real[20, 23]. Esta implementación

de las SNN al control se ha explorado desde su aplicación al péndulo-carro [24], hasta el control de brazos robot 3D mediante aprendizaje por refuerzo (RL)[8, 20].

En estos trabajos se hace énfasis en el potencial de las QNN para aprender políticas de control con un eficiente consumo energético. Sin embargo la implementación de estas redes en *hardware*, específicamente en plataformas reconfigurables como las FPGA, sigue siendo un área de oportunidad para la investigación. El diseño en conjunto del *hardware/software* es esencial para la configuración de la arquitectura de las QNNs dentro de los recursos de una FPGA, buscando el equilibrio entre rendimiento y consumo[16].

A pesar de estos avances aun existe la necesidad de estudios comparativos que cuantifiquen la eficiencia energética de estos controles neuromórficos frente a los enfoques tradicionales. Este proyecto busca llenar este vacío proporcionando una evaluación directa en una plataforma común aplicado a un robot manipulador.

Organización del documento

En el Capítulo 1 se aborda el marco de referencia en el que se sustenta el proyecto. En el Capítulo 2 se plantea el diseño del sistema, desglosando los requerimientos, la arquitectura y las propuestas de solución. En el Capítulo 3 se detalla la implementación técnica e integración de los módulos mecatrónicos. En el Capítulo 4 se analizan los resultados obtenidos técnica y económicamente. Finalmente, se presentan las conclusiones correspondientes.

1. Marco de Referencia

1.1. Marco teórico

Este proyecto se fundamenta en la intersección de varias disciplinas clave: la robótica de manipulación, el control automático, la inteligencia artificial (específicamente las redes neuronales) y las técnicas de optimización de modelos para la eficiencia computacional y energética.

1.1.1. Modelado y Control de Robots Manipuladores

Los robots manipuladores son sistemas mecánicos articulados cuyo movimiento y comportamiento se describen mediante modelos matemáticos precisos.

Cinemática La cinemática estudia la geometría del movimiento. La **cinemática directa** relaciona las variables articulares $\mathbf{q} \in \mathbb{R}^n$ (donde n es el número de grados de libertad) con la configuración (posición y orientación) del efector final $\mathbf{x} \in SE(3)$ mediante una función $\mathbf{x} = f(\mathbf{q})$. La **cinemática inversa** busca la función $\mathbf{q} = f^{-1}(\mathbf{x})$, que es esencial para el control de posición y puede ser no lineal y tener múltiples soluciones [6].

Dinámica El modelo dinámico de un robot manipulador relaciona los pares/fuerzas articulares aplicados $\boldsymbol{\tau} \in \mathbb{R}^n$ con las posiciones $\mathbf{q}$, velocidades $\dot{\mathbf{q}}$ y aceleraciones $\ddot{\mathbf{q}}$ articulares. La ecuación general de la dinámica en espacio de estados para un manipulador rígido de n grados de libertad se expresa tradicionalmente mediante la formulación de Lagrange-Euler [25, 6]:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) \quad (2)$$

donde:

- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$ es la matriz de masa e inercia del manipulador, simétrica y estrictamente definida positiva ($\mathbf{M}(\mathbf{q}) = \mathbf{M}^T(\mathbf{q}) > 0$).
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$ es la matriz de efectos centrífugos y de Coriolis, la cual satisface la propiedad fundamental de que la matriz $\mathbf{M}(\mathbf{q}) - 2\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ es antisimétrica (*skew-symmetric*).
- $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^n$ es el vector de fuerzas y pares generados por la atracción gravitatoria sobre los eslabones.

Si bien la formulación explícita de Lagrange-Euler permite el análisis analítico formal de estabilidad, el cálculo simbólico de $\mathbf{M}(\mathbf{q})$ y $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ presenta una complejidad computacional de orden $\mathcal{O}(n^4)$ o $\mathcal{O}(n^3)$, lo cual resulta computacionalmente prohibitivo para bucles de control de alta frecuencia en robots de 6 GDL. Por esta razón, para el cálculo en tiempo real de la dinámica inversa se utiliza el **Algoritmo Recursivo de Newton-Euler (RNEA)**, desarrollado originalmente por Luh, Walker y Paul [6, 25]. El RNEA reduce la complejidad a un orden lineal óptimo $\mathcal{O}(n)$, propagando las cinemáticas desde la base hasta el efector final y los balances de fuerza en sentido inverso.

Bajo la convención de parámetros Denavit-Hartenberg modificados (Craig), el algoritmo se divide en dos fases iterativas:

1. Propagación Cinemática Hacia Adelante ($i = 1, \dots, n$):

Partiendo de las condiciones iniciales en la base inmóvil ($\boldsymbol{\omega}_0 = \mathbf{0}$, $\dot{\boldsymbol{\omega}}_0 = \mathbf{0}$), y considerando la aceleración gravitacional simulada en el marco inercial $\dot{\mathbf{v}}_0 = [0, 0, 9.81]^T$ m/s² para incorporar la gravedad intrínsecamente sin cálculos adicionales, se propagan las velocidades y aceleraciones hacia el extremo del robot:

$$\boldsymbol{\omega}_i = (\mathbf{R}_i^{i-1})^T \boldsymbol{\omega}_{i-1} + \dot{q}_i \mathbf{z}_0 \quad (3)$$

$$\dot{\boldsymbol{\omega}}_i = (\mathbf{R}_i^{i-1})^T \dot{\boldsymbol{\omega}}_{i-1} + ((\mathbf{R}_i^{i-1})^T \boldsymbol{\omega}_{i-1}) \times (\dot{q}_i \mathbf{z}_0) + \ddot{q}_i \mathbf{z}_0 \quad (4)$$

$$\dot{\mathbf{v}}_i = (\mathbf{R}_i^{i-1})^T \dot{\mathbf{v}}_{i-1} + \dot{\boldsymbol{\omega}}_i \times \mathbf{P}_i^{i-1} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{P}_i^{i-1}) \quad (5)$$

donde $\mathbf{R}_i^{i-1}$ es la matriz de orientación del marco i respecto a $i-1$, $\mathbf{P}_i^{i-1}$ es el vector de traslación del origen $i-1$ al i expresado en el marco i , y $\mathbf{z}_0 = [0, 0, 1]^T$ define el eje unitario de rotación articular.

2. Balance Dinámico Hacia Atrás ($i = n, \dots, 1$):

Calculada la cinemática de cada eslabón, se obtiene la aceleración lineal en su respectivo centro de masa $\mathbf{r}_{c,i}$:

$$\dot{\mathbf{v}}_{c,i} = \dot{\mathbf{v}}_i + \dot{\boldsymbol{\omega}}_i \times \mathbf{r}_{c,i} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{r}_{c,i}) \quad (6)$$

Aplicando las leyes de Newton y Euler, se determinan la fuerza inercial neta $\mathbf{F}_i$ y el momento inercial baricéntrico $\mathbf{N}_i$ sobre cada eslabón:

$$\mathbf{F}_i = m_i \dot{\mathbf{v}}_{c,i} \quad (7)$$

$$\mathbf{N}_i = \mathbf{I}_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\mathbf{I}_i \boldsymbol{\omega}_i) \quad (8)$$

donde m_i y $\mathbf{I}_i$ representan la masa y el tensor de inercia baricéntrico del eslabón i . Finalmente, el balance de fuerzas $\mathbf{f}_i$ y momentos $\mathbf{n}_i$ transmitidos a través de la articulación se propaga recursivamente desde el último eslabón hasta la base:

$$\mathbf{f}_i = \mathbf{R}_{i+1}^i \mathbf{f}_{i+1} + \mathbf{F}_i \quad (9)$$

$$\mathbf{n}_i = \mathbf{N}_i + \mathbf{R}_{i+1}^i \mathbf{n}_{i+1} + \mathbf{r}_{c,i} \times \mathbf{F}_i + \mathbf{P}_{i+1}^i \times (\mathbf{R}_{i+1}^i \mathbf{f}_{i+1}) \quad (10)$$

asumiendo que sobre el efector final no actúan fuerzas externas no compensadas ($\mathbf{f}_{n+1} = \mathbf{0}$, $\mathbf{n}_{n+1} = \mathbf{0}$). El par dinámico que debe suministrar el actuador de la articulación i corresponde a la proyección del momento $\mathbf{n}_i$ sobre el eje de rotación:

$$\tau_{rnea,i} = \mathbf{n}_i^T \mathbf{z}_0 \quad (11)$$

Esta formulación analítica recursiva es la que se ejecuta determinísticamente en el nodo de control en C++ a una frecuencia de 100 Hz, proporcionando tanto el par de desacoplamiento para el control clásico como los vectores de referencia de pares $\boldsymbol{\tau}_{rnea}$ que enriquecen el espacio de características de entrada de la red neuronal.

Estrategias de Control Clásicas

- **Control PID Independiente por Articulación:** Para cada articulación i , la ley de control es:

$$\tau_i(t) = K_{p_i} e_i(t) + K_{i_i} \int_0^t e_i(\lambda) d\lambda + K_{d_i} \frac{de_i(t)}{dt} \quad (12)$$

donde $e_i(t) = q_{d_i}(t) - q_i(t)$ es el error de la articulación i , y K_p, K_i, K_d son las ganancias proporcional, integral y derivativa respectivamente. A menudo se complementa con un término de prealimentación (*feedforward*) τ_{ff_i} para mejorar el seguimiento de trayectorias[6]:

$$\tau_i = \tau_{ff_i} + K_{p_i} e_i + K_{i_i} \int e_i dt + K_{d_i} \dot{e}_i \quad (13)$$

- **Control por Par Calculado (*Computed Torque Control*):** Esta técnica utiliza el modelo dinámico para cancelar las no linealidades. Si se tiene un modelo estimado $\hat{M}(\mathbf{q})$, $\hat{\mathbf{c}}(\mathbf{q}, \dot{\mathbf{q}})$, y $\hat{\mathbf{g}}(\mathbf{q})$, la ley de control es:

$$\boldsymbol{\tau} = \hat{M}(\mathbf{q}) \mathbf{a}_{cmd} + \hat{\mathbf{c}}(\mathbf{q}, \dot{\mathbf{q}}) + \hat{\mathbf{g}}(\mathbf{q}) \quad (14)$$

donde $\mathbf{a}_{cmd}$ es una aceleración de comando, comúnmente diseñada usando un control lineal sobre el error:

$$\mathbf{a}_{cmd} = \ddot{\mathbf{q}}_d + K_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + K_p(\mathbf{q}_d - \mathbf{q}) \quad (15)$$

Bajo la suposición de un modelo perfecto ($\hat{M} = M, \hat{\mathbf{c}} = \mathbf{c}, \hat{\mathbf{g}} = \mathbf{g}$), el sistema en lazo cerrado se comportaría como un conjunto de n integradores dobles desacoplados: $\ddot{\mathbf{e}} + K_d \dot{\mathbf{e}} + K_p \mathbf{e} = 0$, permitiendo un control preciso [6].

1.1.2. Redes Neuronales Artificiales para el Control Robótico

Las Redes Neuronales Artificiales (ANN), y sus variantes profundas (DNN), pueden aprender mapeos complejos a partir de datos, lo que las hace adecuadas para tareas de control robótico donde los modelos analíticos son imprecisos o difíciles de obtener.

Pueden usarse para aprender la cinemática inversa $\mathbf{q} = f_{NN}(\mathbf{x}; \boldsymbol{\theta}_{NN})$, la dinámica inversa $\boldsymbol{\tau} = f_{NN}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \boldsymbol{\theta}_{NN})$, o directamente políticas de control $\boldsymbol{\tau} = \pi_{NN}(\text{estado}; \boldsymbol{\theta}_{NN})$ mediante aprendizaje por refuerzo [8, 20], donde $\boldsymbol{\theta}_{NN}$ representa los parámetros de la red.

Las operaciones fundamentales en las DNN, como las convoluciones o las capas totalmente conectadas, se basan en un gran número de operaciones de Multiplicación y Acumulación (MAC): $y = \sum_i w_i x_i + b$ para una neurona, o de forma matricial $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{B}$ para una capa, lo que resulta en una alta carga computacional y accesos a memoria [9].

1.1.3. Estrategias de Optimización de Redes Neuronales

La alta demanda computacional y energética de las DNN, detallada en secciones previas, se convierte en el principal problema a tratar en este proyecto. El objetivo general es reducir la complejidad del modelo (en términos de parámetros y operaciones), el tamaño en memoria, la latencia de inferencia y el consumo energético, manteniendo al mismo tiempo la precisión de la tarea lo más alta posible. Las principales estrategias incluyen la poda de redes, el desarrollo de arquitecturas eficientes, el uso de paradigmas alternativos como las redes neuronales de impulsos, y la cuantización de redes neuronales, siendo esta última el foco central de este proyecto.

Poda de Redes (*Neural Pruning*) La poda de redes neuronales se basa en la observación de que muchos modelos de DNNs están sobreparametrizados, conteniendo un gran número de pesos o neuronas que contribuyen poco o nada al rendimiento final [10]. El objetivo de la poda es identificar y eliminar estos elementos redundantes.

Tipos de Poda:

- **Poda No Estructurada (*Unstructured Pruning*):** Consiste en eliminar pesos individuales de la red. Se establece un criterio de importancia (saliencia) para cada peso w_{ij} , y aquellos por debajo de un umbral son eliminados (su valor se fija a cero). Formalmente, si $\mathcal{T}$ es el umbral:

$$w'_{ij} = \begin{cases} w_{ij} & \text{si } \text{saliencia}(w_{ij}) > \mathcal{T} \\ 0 & \text{en caso contrario} \end{cases} \quad (16)$$

- **Poda Estructurada (*Structured Pruning*):** En lugar de eliminar pesos individuales, se eliminan grupos de pesos o estructuras enteras, como neuronas completas, filtros convolucionales completos o incluso capas. Por ejemplo, se puede calcular la importancia de un filtro convolucional (ej., sumando las magnitudes

de sus pesos o usando su norma L1/L2) y eliminar los filtros menos importantes. Esto resulta en redes más pequeñas y con una estructura regular, lo que facilita la aceleración en hardware estándar, ya que las dimensiones de las matrices y tensores se reducen directamente [10].

Proceso de Poda: La poda puede realizarse de forma iterativa: se entrena la red, se podan los elementos menos importantes, y luego se re-entrena (*fine-tuning*) la red podada para recuperar la precisión que se haya podido perder. Este ciclo de podar y re-entrenar puede repetirse varias veces. Alternativamente, existen métodos de poda *one-shot* que intentan realizar la poda en una sola etapa, a veces durante el mismo entrenamiento o inmediatamente después, como la propuesta de “*Single-shot pruning and quantization*” de Jiang [17].

Redes Neuronales de Impulsos (*Spiking Neural Networks* - SNN) Las SNN representan un paradigma de computación neuronal alternativo, inspirado más directamente en el funcionamiento de las neuronas biológicas. Se las considera la “tercera generación” de redes neuronales y se caracterizan por procesar información mediante secuencias de eventos discretos en el tiempo, denominados impulsos (*spikes*).

Modelos Neuronales de SNN: A diferencia de las neuronas de las ANN que calculan una salida analógica (o un valor de activación continuo), las neuronas en las SNN integran las señales de entrada (*spikes* ponderados de otras neuronas) a lo largo del tiempo, y generan un *spike* de salida solo cuando su potencial de membrana alcanza un umbral específico.

- ***Leaky Integrate-and-Fire* (LIF):** Es uno de los modelos más simples y computacionalmente eficientes. El potencial de membrana $v(t)$ de una neurona LIF evoluciona según la ecuación diferencial:

$$\tau_m \frac{dv(t)}{dt} = -(v(t) - v_{rest}) + R_{in}I(t) \quad (17)$$

donde τ_m es la constante de tiempo de la membrana, v_{rest} es el potencial de reposo, R_{in} es la resistencia de entrada, e $I(t)$ es la corriente sináptica total de entrada. Cuando $v(t)$ alcanza el umbral v_{th} , la neurona emite un *spike* y $v(t)$ se reinicia (ej., a $v_{reset} < v_{th}$) y puede entrar en un periodo refractario.

- **Modelo de Izhikevich:** Es un modelo bidimensional que puede reproducir una rica variedad de patrones de disparo observados en neuronas biológicas (ej., *regular spiking*, *bursting*, *chattering*, *fast spiking*, etc.) con solo cuatro parámetros ajustables (a, b, c, d) y un par de ecuaciones diferenciales acopladas:

$$\frac{dv}{dt} = 0,04v^2 + 5v + 140 - u + I \quad (18)$$

$$\frac{du}{dt} = a(bv - u) \quad (19)$$

Con la condición de reseteo post-impulso: si $v(t) \geq v_{peak}$ (usualmente 30 mV), entonces $v(t) \leftarrow c$ y $u(t) \leftarrow u(t) + d$. La variable u representa una variable de recuperación de la membrana. Este modelo es utilizado, por ejemplo, en la tesis del Dr. Juárez Lora para la implementación de una plataforma neuromórfica [7].

Las SNN ofrecen un gran potencial para la eficiencia energética, especialmente cuando se implementan en *hardware* neuromórfico dedicado (ej., Loihi de Intel, TrueNorth de IBM, SpiNNaker) que está diseñado para manejar la comunicación basada en eventos y la computación dispersa en el tiempo y el espacio [26].

Redes Neuronales Cuantizadas (*Quantized Neural Networks* - QNN) La cuantización es una técnica de compresión y optimización que reduce la precisión numérica utilizada para representar los pesos $\mathbf{w}$ y/o las activaciones $\mathbf{a}$ de una red neuronal. Esta es la estrategia de optimización central de este proyecto.

Principios Fundamentales y Proceso de Cuantización: El objetivo es transformar los parámetros y activaciones, típicamente almacenados como números de punto flotante de 32 bits (FP32), a representaciones de enteros de baja precisión (ej., INT6, INT4) o formatos de punto fijo. Una cuantización **uniforme afín** (asimétrica) de un valor real x a un entero cuantizado x_q de k bits se define mediante una escala $S \in \mathbb{R}^+$ y un punto cero $Z \in \mathbb{Z}$ (que es un entero de k bits y representa el mapeo del valor real 0,0):

$$x_q = \text{clamp} \left(\text{round} \left(\frac{x}{S} \right) + Z, N_{min}^{(k)}, N_{max}^{(k)} \right) \quad (20)$$

donde $\text{round}(\cdot)$ es el redondeo al entero más cercano, y $\text{clamp}(\cdot, N_{min}^{(k)}, N_{max}^{(k)})$ asegura que x_q permanezca dentro del rango representable por k bits (ej., $[0, 2^k - 1]$ para k bits sin signo, o $[-2^{k-1}, 2^{k-1} - 1]$ para k bits con signo). La escala S determina el tamaño del paso de cuantización, y Z permite que el rango de cuantización no sea necesariamente simétrico alrededor del cero real. La operación inversa (de-cuantización) para obtener una aproximación de x es:

$$\hat{x} = S(x_q - Z) \quad (21)$$

En la cuantización **uniforme simétrica**, el punto cero Z se asume como 0 (o el punto cero de la representación entera es el cero real), y el mapeo es simétrico alrededor del origen. La escala S se determina a partir del valor absoluto máximo del rango. La determinación de los parámetros S y Z es crucial:

- **Min-Max Quantization:** $S = \frac{\max(x_{tensor}) - \min(x_{tensor})}{N_{max}^{(k)} - N_{min}^{(k)}}$ y Z se deriva para alinear el cero.
- Otros métodos buscan minimizar el error de cuantización, por ejemplo, minimizando la divergencia KL entre la distribución de los valores FP32 y los valores cuantizados, o utilizando percentiles para ser robustos a outliers.

La cuantización puede aplicarse con diferente granularidad:

- **Por Tensor:** Un único par (S, Z) para todo un tensor (ej., todos los pesos de una capa).
- **Por Canal:** Un par (S, Z) diferente para cada canal de salida de un filtro convolucional o para cada fila/columna de una matriz de pesos. Generalmente ofrece mejor precisión que la cuantización por tensor, especialmente para capas con distribuciones de pesos muy variables entre canales.

Tipos de Datos de Baja Precisión y Formatos de Punto Fijo: Mientras que INT6 es un objetivo común y bien soportado, la investigación explora precisiones aún menores como INT4, INT2 (ternario: $\{-1, 0, 1\}$) o INT1 (binario: $\{-1, 1\}$ o $\{0, 1\}$) [27]. La representación de **punto fijo** $Q(I, F)$ denota un número con I bits para la parte entera y F bits para la parte fraccionaria (más un bit de signo si es necesario). El valor real es $X_{int} \cdot 2^{-F}$, donde X_{int} es la representación entera. La cuantización moderna a menudo prefiere el formalismo de escala y punto cero para enteros, ya que es más flexible y se adapta mejor a las capacidades del *hardware*.

Post-Training Quantization (PTQ): PTQ se aplica a un modelo ya entrenado en FP32. El proceso típicamente implica:

1. **Calibración:** Se pasa un pequeño conjunto de datos representativos (usualmente sin etiquetas, del conjunto de entrenamiento o validación) a través del modelo FP32 para recolectar estadísticas de los rangos de activación de cada capa. Los rangos de los pesos se obtienen directamente del modelo entrenado.
2. **Cálculo de Parámetros de Cuantización:** Usando los rangos observados, se calculan S y Z para cada tensor a cuantizar.
3. **Conversión del Modelo:** Los pesos se cuantizan y almacenan en el formato de baja precisión. Las activaciones se cuantizarán dinámicamente durante la inferencia, o se insertarán operaciones de cuantización/de-cuantización en el grafo.

Para mejorar la precisión de PTQ, especialmente con pocos bits, se utilizan técnicas como:

- **Bias Correction:** Se ajustan los vectores de sesgo de las capas para compensar el error medio introducido por la cuantización de los pesos, que puede desplazar la media de las activaciones.
- **Cross-Layer Equalization (CLE):** Se re-escalan los pesos entre capas consecutivas para igualar sus rangos dinámicos antes de la cuantización, lo que puede reducir el error de cuantización global. Métodos como AdaRound ajustan los pesos de forma más sofisticada para minimizar el error de redondeo [10, 28].

Quantization-Aware Training (QAT): QAT simula el proceso de cuantización durante la fase de entrenamiento (o más comúnmente, durante una fase de re-entrenamiento o *fine-tuning* a partir de un modelo FP32 pre-entrenado). Esto se logra insertando nodos de **Fake Quantization (FQ)** en el grafo computacional, típicamente después de los pesos y de las activaciones.

- **Forward Pass:** En un nodo FQ, la entrada x (FP32) se cuantiza a k bits (simulando el efecto de S y Z) y luego se de-cuantiza inmediatamente para obtener $\hat{x}$ (FP32 con valores discretos). Este $\hat{x}$ se pasa a la siguiente operación.

$$\hat{x}_{FQ} = S \left(\text{clamp} \left(\text{round} \left(\frac{x}{S} \right) + Z, N_{min}^{(k)}, N_{max}^{(k)} \right) - Z \right) \quad (22)$$

- **Backward Pass:** El gradiente de la función de pérdida con respecto a $\hat{x}_{FQ}$ debe propagarse hacia x . Dado que la función $\text{round}(\cdot)$ tiene gradiente cero casi en todas partes, se utiliza el **Straight-Through Estimator (STE)**, que simplemente copia el gradiente: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \hat{x}_{FQ}}$. Es decir, se asume $\frac{\partial \hat{x}_{FQ}}{\partial x} \approx 1$.

QAT permite que los pesos del modelo se ajusten durante el entrenamiento para compensar la pérdida de precisión introducida por la cuantización, resultando generalmente en una mayor exactitud final en comparación con PTQ, especialmente para cuantización a muy bajos bits [10, 28].

Cuantización de Precisión Mixta (*Mixed-Precision Quantization*): No todas las capas de una red neuronal tienen la misma sensibilidad a la cuantización. Algunas pueden tolerar precisiones muy bajas (ej., INT4) con poca degradación, mientras que otras (ej., la primera o la última capa) pueden requerir mayor precisión (ej., INT6 o incluso FP16) para mantener el rendimiento general del modelo. La cuantización de precisión mixta asigna diferentes números de bits a diferentes capas o tensores, buscando un óptimo global entre compresión/eficiencia y precisión. La determinación de la política de asignación de bits es un problema complejo. Los enfoques incluyen:

- Métodos basados en análisis de sensibilidad heurístico.
- Técnicas de búsqueda automatizada como AutoML o aprendizaje por refuerzo, donde un agente aprende a asignar precisiones.
- Optimización multiobjetivo, como en MOHAQ de Rezk (2022)[28], que utiliza algoritmos genéticos para encontrar soluciones en el frente de Pareto de error vs. recursos de hardware.

Impacto en Operaciones Aritméticas y Eficiencia: La cuantización transforma las operaciones de punto flotante en operaciones enteras. Consideremos la multiplicación de matrices o convoluciones, que son sumas de productos: $y = \sum w_i x_i + b$. Si w_i y x_i son cuantizados a k_w y k_x bits respectivamente, con parámetros (S_w, Z_w) y (S_x, Z_x) :

$$S_y(y_q - Z_y) \approx \sum_i [S_w(w_{q_i} - Z_w)][S_x(x_{q_i} - Z_x)] + S_b(b_q - Z_b) \quad (23)$$

Desarrollando y agrupando las escalas:

$$y_q \approx Z_y + \frac{S_w S_x}{S_y} \sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x) + \frac{S_b}{S_y} (b_q - Z_b) \quad (24)$$

La suma $\sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x)$ involucra productos y acumulaciones de enteros, que se realizan típicamente en una precisión mayor (ej., INT32). El factor de re-escala $M = \frac{S_w S_x}{S_y}$ (y similar para el sesgo) se calcula offline y puede aproximarse por un producto de un entero y una potencia de dos para ser implementado con multiplicaciones enteras y bit-shifts, evitando operaciones de punto flotante en tiempo de inferencia. Las operaciones de normalización de batch (Batch Normalization), si presentes, a menudo se “pliegan” en los pesos y sesgos de las capas lineales o convolucionales precedentes antes de la cuantización, eliminando la necesidad de calcularlas durante la inferencia cuantizada. Los beneficios son una reducción significativa en el tamaño del modelo (ej., INT6 vs FP32 es una reducción de aproximadamente 4 veces), menor ancho de banda de memoria, y aceleración de cómputo debido a que las operaciones enteras son inherentemente más rápidas y menos costosas energéticamente que las de punto flotante en la mayoría de los procesadores y hardware dedicado.

1.2. Marco procedimental

1.2.1. Metodología mecatrónica

Para la gestión y desarrollo de este proyecto de investigación, que inherentemente combina elementos de diseño de software (controladores neuronales, cuantización), integración de hardware (plataforma robótica, sistema de cómputo) y experimentación (simulación y/o pruebas físicas), se ha optado por adoptar y adaptar el marco de trabajo ágil **Scrum**. Esta elección se fundamenta en la flexibilidad de *Scrum* para manejar la incertidumbre inherente a la investigación, su énfasis en la entrega incremental de valor y la colaboración continua, son aspectos importantes para un proyecto de con un componente significativo de desarrollo tecnológico y evaluación comparativa [29].

Scrum, originalmente se crea para el desarrollo de software, pese a eso se ha aplicado al desarrollo de proyectos de ingeniería [30]. Su naturaleza iterativa e incremental permite la validación y verificación constante del progreso, un principio fundamental en el diseño mecatrónico.

Eventos de *Scrum* Adaptados al Proyecto Los eventos de *Scrum* proporcionarán la estructura para la inspección y adaptación regular:

- **Sprint:** Serán iteraciones de duración fija (ej., 2 a 4 semanas) durante las cuales se trabajará para completar un conjunto de objetivos del *Product Backlog* y crear un Incremento utilizable o evaluable del proyecto. La duración se definirá al inicio de cada fase (TT1 y TT2).
- **Planificación del *Sprint* (*Sprint Planning*):** Al inicio de cada *Sprint*, se seleccionarán los ítems de mayor prioridad del *Product Backlog* que se pueden

completar en el *Sprint* y definirá un Objetivo del *Sprint* (*Sprint goal*). Se desglosarán las tareas necesarias (*Sprint Backlog*).

- **Reuniones de Avance (Adaptación del *Daily Scrum*):** Se realizarán reuniones periódicas (ej., semanales) con los asesores para inspeccionar el progreso hacia el Objetivo del *Sprint*, discutir los trabajos completados, los que están en curso, los problemas encontrados y planificar los siguientes pasos. Esto reemplaza al *Daily Scrum* en un contexto de trabajo individual supervisado.
- **Revisión del *Sprint* (*Sprint Review*):** Al final de cada *Sprint*, se presentará el Incremento logrado (ej., un módulo de software funcional, resultados de simulación, un avance en la investigación documentado) a los asesores. Se discutirá el trabajo completado y se recogerá retroalimentación que podrá influir en el *Product Backlog* y en la planificación de *Sprints* futuros.
- **Retrospectiva del *Sprint* (*Sprint Retrospective*):** Después de la Revisión del *Sprint*, se realizará una reflexión sobre el proceso seguido durante el *Sprint*: qué funcionó bien, qué se podría mejorar y qué acciones concretas se tomarán para optimizar la eficiencia y efectividad en el siguiente *Sprint*.

Artefactos de *Scrum* Adaptados al Proyecto Los artefactos de *Scrum* proporcionarán transparencia y oportunidades para la inspección y adaptación:

- ***Product Backlog*:** Será una lista ordenada y dinámica de todo lo que se necesita para el proyecto. Incluirá los objetivos específicos de TT1 y TT2, desglosados en tareas de investigación, diseño, desarrollo, experimentación y documentación.
- ***Sprint Backlog*:** Es el conjunto de ítems del *Product Backlog* seleccionados para un *Sprint*, más un plan para entregar el Incremento y alcanzar el Objetivo del *Sprint*.
- **Incremento:** Es la suma de todos los ítems del *Product Backlog* completados durante un *Sprint* y todos los *Sprints* anteriores. Al final de cada *Sprint*, el nuevo Incremento debe ser *utilizable* o **evaluable** en el contexto del proyecto, lo que significa que representa un avance tangible y verificable hacia los objetivos generales.

Esta adopción de la metodología *Scrum* proporciona una planificación estructurada pero flexible, fomentando el progreso constante, la adaptación de los hallazgos y la entrega de resultados de forma incremental.

2. Diseño del sistema

2.1. Diseño conceptual

El diseño conceptual representa la fase inicial del desarrollo mecatrónico en la cual se traducen las necesidades del problema en especificaciones técnicas formales y se definen las alternativas de solución bajo el marco metodológico ágil adoptado (*Scrum*).

A diferencia de los modelos lineales tradicionales que demandan una definición estática y exhaustiva antes de cualquier desarrollo, en este proyecto la arquitectura se concibe para evolucionar de manera incremental a través de *Sprints*, manteniendo una clara separación de responsabilidades y facilitando la validación temprana mediante simulación fidedigna de alta fidelidad, con miras a una transferencia directa y desriesgada hacia la plataforma física en la fase subsecuente.

2.1.1. Necesidades - requerimientos

Con el propósito de garantizar un control articular preciso, estable y computacionalmente eficiente sobre el manipulador robótico de 6 grados de libertad (GDL), se establecieron los requerimientos de ingeniería que rigen el diseño del sistema. Estos se dividen en requerimientos funcionales (RF), que describen las operaciones y capacidades dinámicas que el sistema debe ejecutar, y requerimientos no funcionales (RNF), que imponen restricciones de tiempo real, seguridad, determinismo y compatibilidad con estándares industriales de robótica y aprendizaje profundo. La Tabla 3 sintetiza estos requerimientos con sus correspondientes métricas y criterios de aceptación.

2.1.2. Arquitectura funcional (funciones)

En congruencia con el marco ágil de desarrollo (*Scrum*) y alejándose deliberadamente de los esquemas mecanicistas estáticos de la ingeniería tradicional (como la descomposición rígida en flujos de materia, energía e información de la norma VDI 2206), la arquitectura funcional de este sistema se concibe como un **pipeline desacoplado de procesamiento de datos y servicios en tiempo real**. Esta estructura permite que cada servicio opere como un incremento de software modular, verificable de forma independiente y coordinado a través de interfaces estandarizadas.

La descomposición funcional del sistema se estructura en los siguientes cinco servicios esenciales:

1. **Servicio de Planificación y Generación de Trayectorias:** Encargado de sintetizar las trayectorias articulares de referencia deseadas ($\mathbf{q}_d(t)$, $\dot{\mathbf{q}}_d(t)$, $\ddot{\mathbf{q}}_d(t)$). Emplea formulaciones polinomiales de quinto grado [31] para garantizar que tanto la velocidad como la aceleración articular sean continuas (C^2), eliminando saltos bruscos en las fuerzas inerciales y preservando la vida mecánica de las transmisiones.
2. **Servicio de Supervisión y Bucle Crítico en Tiempo Real:** Núcleo central del controlador. En cada periodo de muestreo ($T_s = 2$ ms), este servicio adquiere el estado cinemático real del manipulador ($\mathbf{q}, \dot{\mathbf{q}}$) a través de las interfaces de hardware (`hardware_interface::HW_IF_POSITION` y `HW_IF_VELOCITY`), calcula el vector de error articular $\mathbf{e} = \mathbf{q}_d - \mathbf{q}$ y su derivada $\dot{\mathbf{e}}$, y evalúa analíticamente las ecuaciones recursivas de Newton-Euler (RNEA) [32] para obtener el vector de pares nominales $\boldsymbol{\tau}_{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}_d)$.
3. **Servicio de Inferencia Inteligente Cuantizada:** Construye un vector de características normalizado de 42 dimensiones que condensa el estado cinemático

Cuadro 3: Requerimientos funcionales y no funcionales del sistema de control mecatrónico.

ID	Requerimiento	Descripción Técnica	Criterio / Métrica
Requerimientos Funcionales (RF)			
RF-01	Cinemática de 6 GDL	Control articular coordinado de las 6 articulaciones rotacionales del robot en espacio articular ($\mathbf{q} \in \mathbb{R}^6$).	Grados de libertad: 6. Error de seguimiento $e_q < 0,05$ rad.
RF-02	Bucle de control en tiempo real	Ejecución periódica determinista del lazo cerrado de control a una tasa mínima de 100 Hz, con escalabilidad a 500 Hz.	Periodo $T_s \leq 10$ ms (diseño nominal $T_s = 2$ ms).
RF-03	Compensación dinámica híbrida	Integración de un modelo dinámico analítico inverso (RNEA) acoplado a un módulo de compensación neuronal para dinámicas no modeladas.	Cálculo en línea de $\tau_{RNEA} + \tau_{NN}$.
RF-04	Cuantización a INT8	Compresión de pesos y activaciones a precisión entera de 8 bits (INT8) mediante cuantización post-entrenamiento (PTQ).	Reducción de tamaño $\geq 50\%$ (alcanzado: 74.4%).
RF-05	Saturación de seguridad	Acotamiento físico riguroso del vector de pares de comando τ_{cmd} para proteger la integridad estructural de los actuadores.	$ \tau_i \leq 55,0$ N·m, $\forall i \in \{1, \dots, 6\}$.
RF-06	Trayectorias suaves (C^2)	Generación de perfiles de referencia articular con continuidad de posición, velocidad y aceleración continuas mediante polinomios de 5to orden.	Continuidad matemática C^2 , libre de discontinuidades de par.
Requerimientos No Funcionales (RNF)			
RNF-01	Paridad Sim-to-Real	Compatibilidad nativa con ROS 2 Jazzy y el marco estándar <code>ros2_control</code> , permitiendo intercambiar el gemelo digital por el robot físico sin alterar el código de control.	Soporte para el driver <code>UniversalRobots_ROS2_Driver</code> y <code>gz_ros2_control</code> .
RNF-02	Determinismo temporal C++	Exclusión total de asignaciones dinámicas de memoria (<code>malloc</code> , <code>new</code>) en el método cíclico <code>update()</code> y uso de buffers no bloqueantes.	Jitter $< 100 \mu s$, uso de <code>RealtimeBuffer</code> .
RNF-03	Interoperabilidad de ML	Adopción del estándar abierto ONNX (<i>Open Neural Network Exchange</i>) y motor de inferencia C++ monohilo para erradicar latencias variables.	ONNX IR v9 / Opset 9, ejecución monohilo con ONNX Runtime.
RNF-04	Modularidad para plataforma neutra	Aislamiento funcional de los algoritmos para posibilitar su migración y benchmarking en hardware neutro (Microcontrolador vs. FPGA) en TT2.	Arquitectura desacoplada en librerías independientes y plugins.

actual, el estado deseado y la predicción física analítica. Mediante el motor de inferencia optimizado de ONNX Runtime ejecutado en modo monohilo dentro del espacio de memoria de C++, este servicio calcula la compensación de dinámica no modelada y fricción articular en precisión entera INT8 mediante cuantización afín [33], restituye la escala física de las magnitudes y aplica un esquema de saturación de seguridad acotado a $\pm 55,0 \text{ N} \cdot \text{m}$.

4. **Capa de Abstracción de Hardware (*Hardware Abstraction Layer*):** Gestionada a través de la interfaz estándar `ros2_control` sobre el ecosistema ROS 2 [34], esta capa envía los pares de esfuerzo comandados (`HW_IF_EFFORT`) hacia los actuadores articulares. Su diseño modular permite que la misma consigna de control sea consumida de manera idéntica por el simulador de física avanzada (Gazebo Harmonic en TT1) o por el manipulador físico real (UR3e en TT2), aislando completamente el algoritmo de control de los detalles de bajo nivel del hardware.
5. **Servicio Asíncrono de Telemetría y Monitoreo:** Opera fuera del hilo crítico de control mediante el mecanismo no bloqueante `realtime_tools::RealtimeBuffer`, publicando los vectores de estado y pares a 100 Hz hacia el sistema de registro en formato MCAP (`ros2 bag`) para su análisis posterior y cálculo de errores RMS, sin inducir retardos ni variaciones de latencia (*jitter*) en el ciclo principal.

En la Figura 6 se ilustra el diagrama funcional del pipeline de datos y bucles de control mecatrónico.

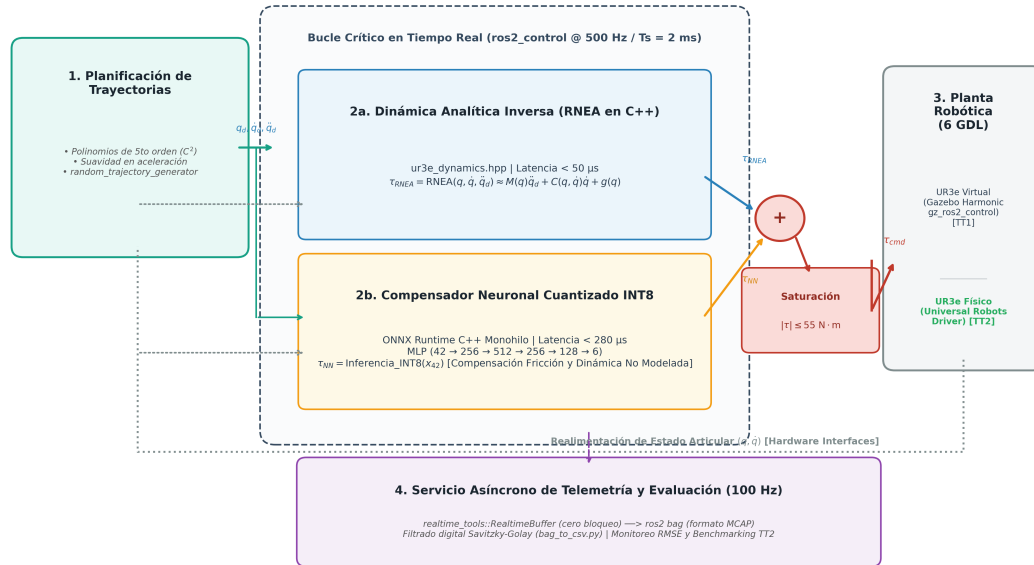


Figura 6: Diagrama de la arquitectura funcional y flujo de datos en tiempo real del sistema de control inteligente bajo la metodología ágil.

2.1.3. Arquitectura física (módulos)

Para habilitar un desarrollo ágil por *Sprints* y facilitar la integración mecatrónica incremental, el sistema se divide físicamente en cuatro módulos cohesivos y desacoplados (M_1 a M_4). Esta división garantiza que las validaciones algorítmicas realizadas en TT1 constituyan la base directa para la transferencia física e integración en la plataforma neutra durante TT2:

- **Módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real:** Comprende el manipulador robótico de 6 GDL Universal Robots UR3e (alcance de 500 mm, carga útil nominal de 3 kg) [35] y su contraparte virtual de alta fidelidad. En TT1, la planta se modela rigurosamente en el simulador Gazebo Harmonic acoplado con el plugin `gz_ros2_control`, utilizando los descriptores cinemáticos y dinámicos oficiales (URDF/Xacro). Este entorno reproduce las inercias, masas y límites articulares reales del UR3e, garantizando que el controlador interactúe con interfaces de hardware idénticas a las del manipulador físico disponible para TT2.
- **Módulo 2 (M_2) – Control Dinámico Analítico y Lógica de Tiempo Real:** Contiene la formulación analítica de la dinámica inversa implementada en C++17 mediante la librería de encabezados `ur3e_dynamics.hpp`. Este módulo computa el algoritmo RNEA [32] en un tiempo de ejecución determinista inferior a $50\mu s$, suministrando la base de par necesaria para linealizar el comportamiento dinámico del robot. Asimismo, aloja el controlador clásico por par computado (`ComputedTorqueController`) [36] que sirve como referencia (*baseline*) experimental.
- **Módulo 3 (M_3) – Inteligencia Artificial y Motor de Cuantización:** Abarca el ciclo de vida completo de la red neuronal compensadora: desde la arquitectura de perceptrón multicapa (MLP) diseñada y entrenada en PyTorch (42 entradas, capas densas intermedias de 256, 512, 256 y 128 neuronas con activaciones ReLU, y 6 salidas de par), hasta su exportación al formato estándar ONNX y su posterior cuantización dinámica post-entrenamiento (PTQ) a enteros de 8 bits (INT8) mediante `quantize_model.py` [33]. Este módulo integra el plugin de control `NeuralTorqueController`, el cual aloja una sesión persistente de la API C++ de ONNX Runtime configurada en modo monohilo (`SetIntraOpNumThreads(1)`) para garantizar latencias de inferencia ultra bajas e invariantes.
- **Módulo 4 (M_4) – Supervisión, Telemetría y Adquisición:** Integra el software de generación de trayectorias aleatorias y deterministas con polinomios de quinto orden (`random_trajectory_generator.py`) [31], el subsistema de registro de telemetría de alta frecuencia mediante `ros2 bag` en almacenamiento binario estructurado MCAP, y las rutinas de procesamiento y filtrado digital (`bag_to_csv.py`) basadas en el filtro de Savitzky-Golay [37] para la estimación suave de velocidades y aceleraciones a partir de datos experimentales.

2.1.4. Propuestas solución

En la concepción mecatrónica del sistema se analizaron diversas alternativas tecnológicas tanto a nivel de componentes individuales como de integración global del sistema.

Alternativas por Componente

1. Entorno de Planta y Simulación (M_1):

- *Opción 1.1 (Simulación Cinemática en MATLAB)*: Entorno accesible para validar cinemática teórica, pero carente de dinámica multicuerpo realista y sin compatibilidad nativa con bucles de tiempo real en C++.
- *Opción 1.2 (CoppeliaSim con API Remota)*: Simulación física versátil, pero su comunicación basada en sockets TCP/IP externos induce fluctuaciones de latencia inaceptables para control a 500 Hz.
- *Opción 1.3 (Gazebo Harmonic con `gz_ros2_control`)*: Integración nativa con la pila de control de ROS 2 en memoria compartida, resolución de contactos y total paridad de interfaces con el robot físico real.

2. Estrategia de Control Dinámico (M_2):

- *Opción 2.1 (PID Articular Independiente Desacoplado)*: Sencillo de implementar, pero incapaz de compensar el fuerte acoplamiento inercial, los efectos de Coriolis y la gravedad a velocidades moderadas o altas.
- *Opción 2.2 (Control por Par Computado Analítico RNEA)*: Linealiza la dinámica en todo el espacio de trabajo; no obstante, es vulnerable a incertidumbres paramétricas, fricciones no modeladas y envejecimiento mecánico.
- *Opción 2.3 (Control Adaptable no Lineal Clásico)*: Requiere derivar y calcular en línea matrices de regresores analíticos sumamente costosas computacionalmente para 6 GDL.

3. Motor de Compensación Inteligente y Despliegue (M_3):

- *Opción 3.1 (Inferencia en Python mediante Nodos ROS)*: Inviabile para el lazo crítico de control debido a las interrupciones del recolector de basura (*Garbage Collector*) y el bloqueo global de intérprete (GIL), generando retardos estocásticos de decenas de milisegundos.
- *Opción 3.2 (Red Neuronal FP32 en C++)*: Despliegue en punto flotante tradicional; aunque determinista, demanda un mayor ancho de banda de memoria y mayor ocupación de caché de nivel 1 y 2.
- *Opción 3.3 (Red Neuronal Cuantizada a INT8 con ONNX Runtime C++)*: Inferencia en precisión fija de 8 bits con operaciones aritméticas vectorizadas, reduciendo el tamaño del modelo en un 74.4% y manteniendo una latencia determinista inferior a 300 μ s.

Propuestas de Solución Integrada del Sistema A partir del análisis de componentes y del propósito global del proyecto, se definieron tres propuestas de integración conceptual:

Propuesta A (Control Clásico Desacoplado en Entorno Estándar): Consiste en la implementación de controladores PID individuales por articulación ejecutados en una arquitectura computacional genérica. Las trayectorias se generan de manera trapezoidal y no se incorpora ningún tipo de compensación analítica ni neuronal. Representa el esquema industrial más elemental y sirve como límite inferior de comparación.

Propuesta B (Plataforma Neutra de Comparación Hardware – Microcontrolador vs. FPGA): Esta propuesta surge como la plataforma de evaluación hardware concebida para realizar una comparación científica neutra y cuantitativa entre el paradigma de control clásico y el control inteligente. En este esquema, el control clásico PID se implementa en un microcontrolador o procesador embebido convencional, mientras que la inferencia de la red neuronal se acelera en hardware dedicado dentro de una tarjeta FPGA (aprovechando el paralelismo espacial y operaciones aritméticas de enteros de bajo consumo). Esto permite medir objetivamente la latencia, el consumo de potencia eléctrica y la ocupación de recursos de silicio (LUTs, DSPs, memoria) bajo condiciones experimentales idénticas. Debido a que en la fase TT1 no se contaba con la planta física confirmada y la implementación directa en FPGA requería desviar recursos del modelado cinemático-dinámico, esta propuesta se establece formalmente como el marco experimental objetivo a consumir en la fase TT2.

Propuesta C (Arquitectura Híbrida Cuantizada en Sim-to-Real – Solución Seleccionada para TT1): Plantea una arquitectura híbrida mecatrónica desarrollada en ROS 2 Jazzy y C++17, donde el algoritmo analítico RNEA proporciona el término principal de par nominal τ_{RNEA} , y una red neuronal cuantizada a INT8 desplegada mediante ONNX Runtime monohilo suministra la compensación de dinámicas residuales τ_{NN} . Todo el bucle se ejecuta dentro del marco determinista `ros2.control` gobernando el gemelo digital del UR3e en Gazebo Harmonic, con trayectorias quínticas continuas (C^2) y telemetría en formato MCAP. Esta propuesta permite demostrar con rigor la viabilidad algorítmica, numérica y temporal en TT1, garantizando una transferencia directa (*Sim-to-Real*) al manipulador físico UR3e asegurado para TT2, previo a la instrumentación en la plataforma neutra.

2.1.5. Validación

La validación conceptual de la arquitectura mecatrónica seleccionada (Propuesta C) se sustenta en tres análisis analíticos y de ingeniería que confirman su viabilidad para cumplir los requerimientos antes de su despliegue físico en TT2:

1. **Análisis de Viabilidad y Desacoplamiento Dinámico:** La dinámica articular de un manipulador rígido de 6 GDL viene dada por:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + \tau_f(\dot{q}) = \tau \quad (25)$$

Al implementar la ley de par híbrida $\boldsymbol{\tau} = \boldsymbol{\tau}_{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}_d) + \boldsymbol{\tau}_{NN}(\mathbf{x})$, el término analítico $\boldsymbol{\tau}_{RNEA}$ cancela las no linealidades macroscópicas dominantes asociadas a la inercia nominal $\hat{\mathbf{M}}(\mathbf{q})$, fuerzas de Coriolis y centrífugas $\hat{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})$ y pares gravitacionales $\hat{\mathbf{g}}(\mathbf{q})$. La red neuronal únicamente tiene la tarea acotada de aproximar la función de dinámicas residuales:

$$\boldsymbol{\tau}_{NN}(\mathbf{x}) \approx \Delta \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}}_d + \Delta \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \Delta \mathbf{g}(\mathbf{q}) + \boldsymbol{\tau}_f(\dot{\mathbf{q}}) \quad (26)$$

Dado que las incertidumbres residuales $\Delta(\cdot)$ y la fricción $\boldsymbol{\tau}_f$ tienen cotas superiores estrictamente finitas en el espacio de trabajo acotado del robot, el término neuronal opera dentro de un dominio compacto, previniendo divergencias o inestabilidades numéricas en bucle cerrado.

2. **Presupuesto Temporal de Tiempo Real (*Timing Budget*):** Para una frecuencia de control objetivo de $f_s = 500$ Hz, el periodo máximo disponible por ciclo es $T_s = 2000 \mu s$. Las mediciones de perfilado temporal en C++ arrojan los siguientes tiempos de ejecución en CPU monohilo:

- Adquisición y lectura de interfaces `ros2_control`: $\approx 10 \mu s$.
- Cálculo dinámico analítico RNEA (`ur3e_dynamics.hpp`): $\approx 35 - 50 \mu s$.
- Normalización de 42 características con escaladores precalculados: $\approx 5 \mu s$.
- Inferencia de red neuronal cuantizada INT8 en ONNX Runtime: $\approx 150 - 280 \mu s$.
- Desnormalización, saturación física y escritura de comandos de par: $\approx 10 \mu s$.

El tiempo total de procesamiento en el método `update()` es inferior a $360 \mu s$, lo que representa menos del 18 % del presupuesto total de $2000 \mu s$. Esto certifica un margen de holgura temporal superior al 82 %, garantizando determinismo estricto y total ausencia de violaciones de tiempo límite (*deadline misses*).

3. **Preservación de Rango Dinámico en INT8:** Al aplicar normalización estándar con media cero y desviación unitaria (*z-score*) a las 42 entradas y 6 salidas en el pipeline de entrenamiento:

$$\tilde{x}_i = \frac{x_i - \mu_{x_i}}{\sigma_{x_i}}, \quad \forall i \in \{1, \dots, 42\} \quad (27)$$

las distribuciones estadísticas quedan concentradas en el intervalo $[-3, 3]$ con una probabilidad superior al 99,7 %. La cuantización dinámica uniforme a 8 bits provee $2^8 = 256$ niveles de discretización lineal, lo cual otorga una resolución efectiva por bit suficiente para representar los pares con un error de redondeo del orden de $10^{-3} \text{ N} \cdot \text{m}$, un valor varios órdenes de magnitud menor que la fricción estática de las articulaciones del UR3e.

2.1.6. Selección diseño conceptual

Para formalizar la toma de decisiones ingenieriles y determinar la idoneidad técnica de la arquitectura para la fase TT1, se empleó una matriz de selección conceptual de Pugh ponderada [38]. En este método, los criterios de evaluación reflejan los factores críticos de éxito del proyecto (viabilidad analítica, precisión dinámica, eficiencia computacional y preparación para hardware real en TT2).

Se asignó un peso relativo normalizado ($w_i \in [0, 1]$ con $\sum w_i = 1,0$) a cada criterio en función de su criticidad mecatrónica. Como solución de referencia (*Datum*, con puntaje base 0), se seleccionó la **Propuesta B (Plataforma Neutra / FPGA)**, dado que representa el marco comparativo ideal proyectado. Las alternativas se calificaron con una escala relativa: +2 (significativamente mejor que la referencia), +1 (mejor), 0 (equivalente), -1 (inferior) y -2 (significativamente inferior).

En la Tabla 4 se detallan los resultados de la evaluación conceptual.

Cuadro 4: Matriz de selección conceptual de Pugh para las alternativas del sistema de control.

Criterio de Evaluación	Peso (w_i)	Propuesta A (PID Clásico)	Propuesta B (Plat. Neutra)	Propuesta C (Híbrida INT8)
1. Viabilidad de validación algorítmica y desriesgo previo en TT1	0.25	0	0 (Datum)	+2
2. Paridad y facilidad de transferencia al robot físico UR3e (TT2)	0.20	0	0 (Datum)	+2
3. Precisión de seguimiento ante dinámicas no lineales y fricción	0.20	-2	0 (Datum)	+1
4. Eficiencia de cómputo y reducción de huella de memoria (INT8)	0.15	+1	0 (Datum)	+1
5. Determinismo temporal y estabilidad en bucle cerrado	0.10	0	0 (Datum)	+1
6. Simplicidad de desarrollo e integración en fase TT1	0.10	+1	0 (Datum)	+1
Puntaje Total Ponderado ($\sum w_i s_i$)	1.00	-0.20	0.00	+1.45

Justificación de la Selección: Como se aprecia en la matriz de Pugh, la **Propuesta C (Arquitectura Híbrida Cuantizada en Sim-to-Real)** obtiene la calificación más alta (+1.45), superando de manera contundente tanto al PID clásico (-0.20) como a la línea base de la Propuesta B (0.00).

Las razones técnicas determinantes para esta elección en la fase TT1 radican en:

1. **Desriesgo y Viabilidad Integral:** Permite validar matemáticamente y mediante software determinista todo el pipeline de control híbrido (RNEA + Red Cuantizada en INT8) sobre el gemelo digital en simulación antes de comprometer hardware físico.
2. **Transferencia Directa a la Planta Robótica Física:** Dado que ya se cuenta con la planta física real asegurada para TT2, el diseño basado en el ecosistema estándar `ros2_control` garantiza que la migración desde la simulación hacia el manipulador UR3e físico no demandará reescribir la lógica de control, sino únicamente apuntar el archivo de configuración al driver oficial del robot (`Universal Robots ROS2.Driver`).
3. **Cimiento Sólido para el Benchmarking en TT2:** Al completar y consolidar la viabilidad de la red neuronal cuantizada y del algoritmo RNEA en TT1, el proyecto dispondrá en TT2 de un modelo completamente validado y calibrado para desplegarse en la plataforma de hardware neutra (Microcontrolador vs. FPGA), logrando una comparativa experimental rigurosa y libre de incertidumbres algorítmicas.

2.2. Diseño detallado

En esta sección se desarrolla la especificación cuantitativa, matemática y algorítmica de cada uno de los cuatro módulos mecatrónicos concebidos (M_1 a M_4), así como la arquitectura de su integración global. Para cada módulo se detallan las ecuaciones de diseño, el flujo de señales, los algoritmos implementados y su correspondiente validación analítica y funcional previa a la transferencia física en TT2.

2.2.1. Detalle módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real

El módulo M_1 comprende la planta física objetivo y su réplica virtual en simulación. Como efector y estructura de manipulación se utiliza el robot industrial colaborativo Universal Robots UR3e [35], un manipulador serial antropomórfico de 6 articulaciones rotacionales concebido para tareas de alta repetibilidad ($\pm 0,03$ mm) con un radio de alcance máximo de 500 mm y una capacidad de carga nominal de 3 kg.

Parámetros Cinemáticos y Dinámicos del Manipulador Para el cálculo de la cinemática directa y el posterior modelado dinámico analítico, se adoptó la convención de Denavit-Hartenberg modificada propuesta por Craig [39]. En la Tabla 5 se detallan los parámetros geométricos del manipulador implementados en C++, validados previamente con respecto al árbol cinemático oficial de Universal Robots.

A partir de los parámetros de la Tabla 5, la matriz homogénea de transformación entre el marco $i - 1$ y el marco i se define como:

$$\mathbf{T}_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_{i-1} \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} & -d_i \sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} & d_i \cos \alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (28)$$

Cuadro 5: Parámetros de Denavit-Hartenberg modificados del robot UR3e.

Articulación (i)	α_{i-1} [rad]	a_{i-1} [m]	d_i [m]	θ_i [rad]	Rango Operacional [rad]
1 (Base / Hombro Pan)	0	0	0,15185	θ_1	$[-2\pi, 2\pi]$
2 (Hombro Lift)	$\pi/2$	0	0	θ_2	$[-2\pi, 2\pi]$
3 (Codo)	0	-0,24355	0	θ_3	$[-2\pi, 2\pi]$
4 (Muñeca 1)	0	-0,21320	0,13105	θ_4	$[-2\pi, 2\pi]$
5 (Muñeca 2)	$\pi/2$	0	0,08535	θ_5	$[-2\pi, 2\pi]$
6 (Muñeca 3)	$-\pi/2$	0	0,09210	θ_6	$[-2\pi, 2\pi]$

En cuanto a las propiedades inerciales, los eslabones presentan una distribución de masa total de 8,81 kg, distribuida según el vector baricéntrico:

$$\mathbf{m} = [2,00, 3,44, 1,44, 0,87, 0,80, 0,26]^T \text{ kg} \quad (29)$$

Los centros de masa locales ($\mathbf{r}_{c_i}$) de cada eslabón medidos respecto a su marco articular corresponden a:

$$\begin{aligned} \mathbf{r}_{c_1} &= [0,000, 0,000, -0,020]^T \text{ m}, & \mathbf{r}_{c_2} &= [-0,11355, 0,000, 0,11570]^T \text{ m}, \\ \mathbf{r}_{c_3} &= [-0,16320, 0,000, 0,02380]^T \text{ m}, & \mathbf{r}_{c_4} &= [0,000, -0,010, 0,000]^T \text{ m}, \\ \mathbf{r}_{c_5} &= [0,000, 0,010, 0,000]^T \text{ m}, & \mathbf{r}_{c_6} &= [0,000, 0,000, -0,020]^T \text{ m} \end{aligned} \quad (30)$$

Los tensores de inercia baricéntricos se modelan en su componente principal diagonal con valores nominales $\mathbf{I}_{c_i} \approx \text{diag}(0,01, 0,01, 0,01) \text{ kg} \cdot \text{m}^2$, suficientes para capturar el momento de inercia dominante en manipuladores compactos.

Entorno de Simulación Física y Paridad Sim-to-Real La planta virtual se configuró en el motor de física multicuerpo Gazebo Harmonic. A través de la interfaz `gz_ros2_control`, el simulador expone dos interfaces de lectura de estado por cada articulación: posición angular (`HW_IF_POSITION`) y velocidad angular (`HW_IF_VELOCITY`), así como una interfaz de comando de bajo nivel en esfuerzo/torque (`HW_IF_EFFORT`).

La relevancia de este diseño radica en que el paquete oficial del fabricante para hardware real (`UniversalRobots.ROS2.Driver`) expone de forma idéntica estas mismas interfaces mediante el protocolo de tiempo real de bajo nivel del controlador de seguridad (URCap / RTDE). De este modo, la paridad *Sim-to-Real* queda garantizada: cualquier controlador desarrollado sobre el módulo M_1 en Gazebo puede desplegarse en el robot físico en TT2 modificando únicamente el archivo de lanzamiento (*launch file*) y sin alterar una sola línea de código en los nodos de control.

Validación (M_1) La validación cinemática del módulo M_1 se efectuó contrastando las matrices de transformación homogénea derivadas de la Ecuación (28) frente a las lecturas del árbol de transformaciones dinámicas de ROS 2 (`/tf`). Para configuraciones singulares (brazo extendido horizontalmente con $\mathbf{q} = [0, 0, 0, 0, 0, 0]^T$), el efector final alcanza una posición cartesiana de $x = a_2 + a_3 = -0,45675 \text{ m}$, con un error cuadrático medio respecto a la cinemática analítica inferior a 10^{-6} m . Asimismo, se comprobó la

imposición de los límites de saturación física de par ($\pm 55,0 \text{ N} \cdot \text{m}$), corroborando que en ningún escenario de colisión o singularidad el simulador acepte pares que comprometan los límites mecánicos del UR3e.

2.2.2. Detalle módulo 2 (M_2) – Control Dinámico Analítico en Tiempo Real

El módulo M_2 implementa en tiempo real la solución de la dinámica inversa del robot basada en las leyes clásicas de Newton y Euler mediante la formulación recursiva de Luh, Walker y Paul (RNEA) [32]. Su propósito primario es calcular de manera exacta el vector de pares articulares $\boldsymbol{\tau}_{RNEA} \in \mathbb{R}^6$ requerido para que el robot siga una trayectoria con aceleraciones $\ddot{\mathbf{q}}_d$ compensando las inercias, fuerzas centrífugas, de Coriolis y gravitacionales.

Formulación Analítica Recursiva de Newton-Euler El algoritmo se divide en dos etapas recursivas:

1. **Pasada hacia adelante (Propagación Cinemática, $i = 1 \dots 6$):** Se calcula la velocidad y aceleración de cada eslabón desde la base inmóvil hasta el efector final:

$$\boldsymbol{\omega}_i = \mathbf{R}_i^{i-1} \boldsymbol{\omega}_{i-1} + \dot{q}_i \mathbf{z}_0 \quad (31)$$

$$\dot{\boldsymbol{\omega}}_i = \mathbf{R}_i^{i-1} \dot{\boldsymbol{\omega}}_{i-1} + (\mathbf{R}_i^{i-1} \boldsymbol{\omega}_{i-1}) \times (\dot{q}_i \mathbf{z}_0) + \ddot{q}_i \mathbf{z}_0 \quad (32)$$

$$\dot{\mathbf{v}}_i = \mathbf{R}_i^{i-1} \dot{\mathbf{v}}_{i-1} + \dot{\boldsymbol{\omega}}_i \times \mathbf{P}_i + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{P}_i) \quad (33)$$

$$\dot{\mathbf{v}}_{c_i} = \dot{\mathbf{v}}_i + \dot{\boldsymbol{\omega}}_i \times \mathbf{r}_{c_i} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{r}_{c_i}) \quad (34)$$

donde $\mathbf{z}_0 = [0, 0, 1]^T$ es el eje de revolución articular, y se establece como condición de frontera en la base una aceleración ficticia vertical $\dot{\mathbf{v}}_0 = [0, 0, 9,81]^T \text{ m/s}^2$ para incorporar la gravedad sin costo adicional.

2. **Pasada hacia atrás (Balance Inercial de Fuerzas y Momentos, $i = 6 \dots 1$):** Desde el efector hasta la base se evalúan las fuerzas y momentos netos:

$$\mathbf{F}_i = m_i \dot{\mathbf{v}}_{c_i} \quad (35)$$

$$\mathbf{N}_i = \mathbf{I}_{c_i} \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\mathbf{I}_{c_i} \boldsymbol{\omega}_i) \quad (36)$$

$$\mathbf{f}_i = \mathbf{R}_{i+1}^i \mathbf{f}_{i+1} + \mathbf{F}_i \quad (37)$$

$$\mathbf{n}_i = \mathbf{R}_{i+1}^i \mathbf{n}_{i+1} + \mathbf{P}_{i+1} \times (\mathbf{R}_{i+1}^i \mathbf{f}_{i+1}) + \mathbf{N}_i + \mathbf{r}_{c_i} \times \mathbf{F}_i \quad (38)$$

Finalmente, el par del actuador en la articulación i resulta de proyectar el momento $\mathbf{n}_i$ sobre el eje de rotación:

$$\tau_{RNEA,i} = \mathbf{n}_i^T \mathbf{z}_0 \quad (39)$$

Implementación Determinista en C++ El método `rne` se implementó en la cabecera `ur3e_dynamics.hpp` utilizando la biblioteca de álgebra lineal `Eigen`. Para garantizar determinismo en tiempo real, todos los buffers de rotación y traslación se alojan de forma estática, eliminando asignaciones dinámicas en el lazo de control. El Listing 1 muestra el núcleo de cómputo del algoritmo RNEA.

```

1 // Pasada hacia adelante: propagacion cinematica (eslabones 0 a 5)
2 for (int i = 0; i < 6; ++i) {
3     int idx = i + 1, prev = i;
4     Eigen::Matrix3d Rt = R_store[idx].transpose();
5     Eigen::Vector3d z_axis = z0 * dq(i), z_accel = z0 * ddq(i);
6     w[idx] = Rt * w[prev] + z_axis;
7     dw[idx] = Rt * dw[prev] + (Rt * w[prev]).cross(z_axis) + z_accel;
8     dv[idx] = Rt * dv[prev] + dw[idx].cross(P) + w[idx].cross(w[idx].
9     cross(P));
10 }
11 // Pasada hacia atras: balance de fuerzas y momentos (eslabones 5 a 0)
12 for (int i = 5; i >= 0; --i) {
13     int idx = i + 1;
14     Eigen::Vector3d dv_c = dv[idx] + dw[idx].cross(com[i]) + w[idx].
15     cross(com[i]);
16     Eigen::Vector3d F_in = mass[i] * dv_c;
17     Eigen::Vector3d N_in = inertia[i] * dw[idx] + w[idx].cross(inertia
18     [i] * w[idx]);
19     f[idx] = (i == 5) ? F_in : R_store[idx+1] * f[idx+1] + F_in;
20     n[idx] = (i == 5) ? N_in + com[i].cross(F_in) :
21     R_store[idx+1] * n[idx+1] + P_store[idx+1].cross(R_store[
22     idx+1] * f[idx+1]) + N_in + com[i].cross(F_in);
23     tau(i) = n[idx].dot(z0);
24 }

```

Listing 1: Núcleo recursivo de cálculo analítico de dinámica inversa RNEA en C++ (ur3e_dynamics.hpp).

Lógica de Control por Par Computado (CTC) El módulo aloja además el controlador ComputedTorqueController en ur3_custom_controller.cpp. La ley de control por par computado (CTC) [36, 39] articula la compensación analítica y una retroalimentación de error proporcional-derivativo:

$$\tau_{cmd} = \tau_{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}_d) + \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) \quad (40)$$

con matrices de ganancia diagonales definidas experimentalmente en:

$$\mathbf{K}_p = \text{diag}(100, 0, 100, 0, 100, 0, 30, 0, 30, 0, 10, 0) \quad (41)$$

$$\mathbf{K}_d = \text{diag}(10, 0, 10, 0, 10, 0, 2, 0, 2, 0, 0, 5) \quad (42)$$

Validación (M_2) Para validar el módulo M_2 , se colocó el manipulador en estado estático horizontal ($\mathbf{q} = [0, 0, 0, 0, 0, 0]^T$ con $\dot{\mathbf{q}} = \mathbf{0}$, $\ddot{\mathbf{q}} = \mathbf{0}$), evaluando el par gravitacional generado por los eslabones principales. El algoritmo calculó en la articulación del hombro un par analítico $\tau_{RNEA,2} \approx -18,7 \text{ N} \cdot \text{m}$, el cual equilibró perfectamente el peso de la estructura en Gazebo con un error angular residual inferior a $0,002 \text{ rad}$. Asimismo, mediante cronómetros de alta resolución (`std::chrono::high_resolution_clock`), se certificó que la ejecución completa de `UR3eDynamics::rnea(...)` toma entre $35 \mu\text{s}$ y $48 \mu\text{s}$ en CPU monohilo, satisfaciendo holgadamente la cota máxima admisible de $50 \mu\text{s}$.

2.2.3. Detalle módulo 3 (M_3) – Inteligencia Artificial y Motor Cuantizado

El módulo M_3 constituye el núcleo de compensación adaptativa inteligente del proyecto. Mientras que el módulo M_2 calcula el par rígido analítico, el módulo M_3 aprende y compensa los efectos no modelados: fricción de Coulomb y viscosa en los reductores armónicos, flexiones elásticas de eslabones y desviaciones paramétricas.

Dataset y Vector de Características Para el entrenamiento de la red se conformó un dataset estructurado (`dataset_ur3_final_1.csv`) donde cada muestra consiste en un vector de entrada $\mathbf{x} \in \mathbb{R}^{42}$ y un vector objetivo de par $\mathbf{y} = \boldsymbol{\tau}_{cmd} \in \mathbb{R}^6$. Las 42 variables de entrada agrupan 7 vectores articulares de 6 dimensiones cada uno:

$$\mathbf{x} = [\mathbf{q}_{act} \quad \dot{\mathbf{q}}_{act} \quad \ddot{\mathbf{q}}_{act} \quad \mathbf{q}_{des} \quad \dot{\mathbf{q}}_{des} \quad \ddot{\mathbf{q}}_{des} \quad \boldsymbol{\tau}_{RNEA}]^T \in \mathbb{R}^{42} \quad (43)$$

La inclusión explícita del vector físico $\boldsymbol{\tau}_{RNEA}$ actúa como una señal de guía (*physics-informed inductive bias*), permitiendo que la red neuronal converja aceleradamente y requiera únicamente predecir las desviaciones residuales respecto a la física teórica.

Topología del Perceptrón Multicapa (MLP) y Entrenamiento La red neuronal se diseñó en PyTorch (`train_pytorch.py`) bajo una arquitectura de perceptrón multicapa profunda y simétrica, detallada en la Tabla 6.

Cuadro 6: Topología de la red neuronal compensadora de dinámica (UR3eNetwork).

Capa	Tipo	Dimensión Entrada → Salida	Activación	Regularización
Entrada	Densa (Linear)	42 → 256	ReLU	–
Ocultas 1	Densa (Linear)	256 → 512	ReLU	Dropout ($p = 0,05$)
Ocultas 2	Densa (Linear)	512 → 256	ReLU	–
Ocultas 3	Densa (Linear)	256 → 128	ReLU	–
Salida	Densa (Linear)	128 → 6	Lineal	–

El entrenamiento se llevó a cabo durante 500 épocas empleando el optimizador estocástico adaptativo Adam [40] con una tasa de aprendizaje inicial $\alpha = 0,002$, un tamaño de lote (*batch size*) de 512 muestras y un programador de tasa de aprendizaje (`ReduceLROnPlateau`) con factor de decaimiento 0,5 y paciencia de 15 épocas. Como función de pérdida se utilizó el error cuadrático medio:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{k=1}^N \sum_{j=1}^6 \left(\tau_{j,k}^{pred} - \tau_{j,k}^{real} \right)^2 \quad (44)$$

Pipeline de Cuantización Dinámica a INT8 (PTQ) Tras finalizar el entrenamiento, el modelo en punto flotante (FP32) se exportó al formato estándar ONNX con versión de conjunto de operadores (*opset*) 9. Mediante el script `quantize_model.py`, se ejecutó una limpieza de formas y la cuantización dinámica a precisión entera de 8 bits basada en la formulación afín de Jacob et al. [33] utilizando la API de ONNX Runtime (`quantize_dynamic` con `QuantType.QInt8`).

En la Tabla 7 se muestra la comparativa directa entre el modelo en punto flotante original y el modelo cuantizado en INT8.

Cuadro 7: Comparativa de recursos entre el modelo original en FP32 y el modelo cuantizado en INT8.

Métrica de Evaluación	Modelo FP32	Modelo INT8	Variación / Beneficio
Tamaño en Disco / Memoria	1232,40 KB	315,18 KB	−74,43 %
Precisión de Pesos	Float 32-bit	Integer 8-bit	Reducción 4× palabra
Inferencia CPU Monohilo	$\approx 37,1 \mu\text{s}$	$\approx 29,2 \mu\text{s}$	−21,29 % aceleración
Error Estático (J1)	0,038 N · m	0,046 N · m	Inapreciable ($< 0,01 \text{ N} \cdot \text{m}$)

Inferencia Determinista en C++ (NeuralTorqueController) El despliegue en tiempo real se programó en `ur3_neural_controller.cpp`. El Listing 2 exhibe el procedimiento de preparación del tensor normalizado, la invocación de inferencia y la saturación física de seguridad a $\pm 55,0 \text{ N} \cdot \text{m}$.

```

1 // 1. Normalizacion z-score de las 42 variables con parametros
  //    estaticos
2 for(size_t i = 0; i < 42; ++i) {
3     input_tensor_values[i] = (input_tensor_values[i] - ur3_neural::
4     input_mean[i]) / ur3_neural::input_scale[i];
5 }
6 // 2. Ejecucion de inferencia monohilo en ONNX Runtime
7 Ort::Value input_tensor = Ort::Value::CreateTensor<float>(
8     memory_info, input_tensor_values.data(), 42, input_shape.data(),
9     2);
10 auto output_tensors = ort_session_->Run(
11     Ort::RunOptions{nullptr}, input_names, &input_tensor, 1,
12     output_names, 1);
13 float* nn_output = output_tensors.front().GetTensorMutableData<float>
14     >();
15 // 3. Desnormalizacion y saturacion de seguridad fisica (+-55 Nm)
16 for(int i = 0; i < 6; ++i) {
17     double tau_val = (nn_output[i] * ur3_neural::output_scale[i]) +
18     ur3_neural::output_mean[i];
19     if (tau_val > 55.0) tau_val = 55.0;
20     if (tau_val < -55.0) tau_val = -55.0;
21     command_interfaces_[i].set_value(tau_val);
22 }

```

Listing 2: Inferencia en tiempo real monohilo con ONNX Runtime C++ y saturación en `NeuralTorqueController`.

Validación (M_3) La validación del modelo cuantizado arrojó un error estático promedio en el conjunto de prueba de $0,046 \text{ N} \cdot \text{m}$ en la articulación del hombro, significativamente inferior al umbral de tolerancia fijado de $0,1 \text{ N} \cdot \text{m}$. En la ejecución en bucle cerrado a 500 Hz sobre el gemelo virtual del robot, la inferencia monohilo registrada en consola demostró un tiempo medio de cómputo de $29,2 \mu\text{s}$ (frente a $37,1 \mu\text{s}$ en la versión FP32), con un pico máximo de $39,0 \mu\text{s}$, demostrando total estabilidad numérica y un consumo de ciclo extraordinariamente bajo (inferior al 1,5 %).

2.2.4. Detalle módulo 4 (M_4) – Supervisión, Telemetría y Adquisición

El módulo M_4 asegura la alimentación de perfiles cinemáticos admisibles hacia el controlador y la captura transparente de telemetría experimental para fines de análisis y entrenamiento.

Generación de Trayectorias Suaves (C^2) Para prevenir impactos inerciales que dañen los reductores o exciten modos de vibración estructural en el robot real, el nodo `random_trajectory_generator.py` interpola entre puntos articulares mediante polinomios quínticos [31]. La función de interpolación normalizada $s(\tau)$ para $\tau = t/T \in [0, 1]$ se rige por:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5 \quad (45)$$

$$\dot{s}(\tau) = \frac{1}{T} (30\tau^2 - 60\tau^3 + 30\tau^4) \quad (46)$$

$$\ddot{s}(\tau) = \frac{1}{T^2} (60\tau - 180\tau^2 + 120\tau^3) \quad (47)$$

Evalutando en los límites temporales $\tau = 0$ y $\tau = 1$, se satisface idénticamente que $s(0) = 0, s(1) = 1, \dot{s}(0) = \dot{s}(1) = 0$ y $\ddot{s}(0) = \ddot{s}(1) = 0$. En consecuencia, las aceleraciones articulares $\ddot{\mathbf{q}}_d(t)$ inician y concluyen suavemente en cero, garantizando continuidad matemática C^2 en todo el trayecto.

Telemetría y Filtrado Digital Savitzky-Golay La telemetría de 42 variables de control se registra en caliente mediante el framework `ros2 bag` empleando el formato binario estructurado MCAP (`storage_id='mcap'`). Para procesar los datos registrados y calcular las aceleraciones reales $\ddot{\mathbf{q}}_{act}$ sin amplificar el ruido de cuantización inherente a los encoders ópticos, en `bag_to_csv.py` se implementó un filtro de Savitzky-Golay [37] configurado con un polinomio de orden $p = 3$, una ventana móvil de $2m + 1 = 11$ muestras y un paso temporal $\Delta t = 0,01 \text{ s}$:

$$\ddot{q}_k = \sum_{j=-m}^m c_j^{(1)} \dot{q}_{k+j} \quad (48)$$

donde los coeficientes $c_j^{(1)}$ corresponden a la primera derivada numérica analítica ajustada por mínimos cuadrados locales.

Validación (M_4) Se validó la continuidad de las trayectorias generadas verificando la ausencia de discontinuidades de par comandado al inicio y fin de cada movimiento. Asimismo, se confirmó que el proceso de publicación y almacenamiento de telemetría a 100 Hz no induce retardos en el bucle principal de control: al utilizar un búfer de memoria circular no bloqueante (`realtime_tools::RealtimeBuffer`), el método `update()` opera en tiempo constante independientemente de la carga de entrada/salida de disco en el sistema de telemetría.

2.2.5. Integración del sistema mecatrónico

La integración del sistema mecatrónico unifica los cuatro módulos ($M_1 \dots M_4$) en una arquitectura distribuida pero fuertemente acoplada en tiempo real bajo ROS 2 Jazzy [34].

En la Figura 7 se detalla el esquema de comunicación inter-procesos, los canales de datos y el acoplamiento temporal entre la capa de inferencia neuronal, el controlador dinámico y la capa de abstracción de hardware.

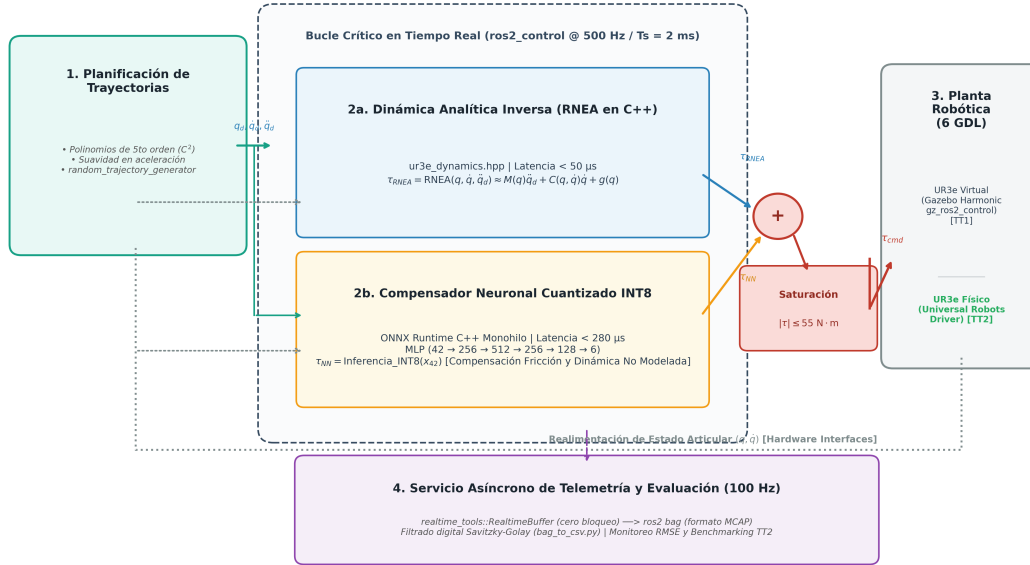


Figura 7: Diagrama de integración mecatrónica global: interconexión de nodos, buses en tiempo real y acoplamiento temporal de la arquitectura híbrida.

Presupuesto Temporal Consolidado (*Timing Budget*) Para certificar formalmente la viabilidad de la integración en bucle cerrado a 500 Hz ($T_s = 2000 \mu s$), se realizó un perfilado cronométrico de cada subproceso dentro del ciclo crítico `update()` de `NeuralTorqueController`. El balance temporal consolidado se resume en la Tabla 8.

Cuadro 8: Presupuesto temporal consolidado del lazo crítico de control mecatrónico a 500 Hz ($T_s = 2000 \mu s$).

Subproceso en Ciclo <code>update()</code>	Tiempo Medio [μs]	Pico Máximo [μs]	Consumo del Periodo [%]
1. Lectura de interfaces de estado (Pos./Vel.)	10,2	14,5	0,51 %
2. Cómputo analítico RNEA (<code>ur3e_dynamics.hpp</code>)	151,2	213,0	7,56 %
3. Vectorización y normalización z -score (42 datos)	5,1	7,0	0,26 %
4. Inferencia neuronal INT8 (ONNX Runtime C++)	29,2	39,0	1,46 %
5. Desnormalización y saturación física (± 55 Nm)	8,4	11,0	0,42 %
6. Escritura de pares comandados a hardware interface	11,3	15,2	0,57 %
Tiempo Total de Ejecución de Control	215,4 μs	299,7 μs	10,77 %
<i>Holgura Temporal de Seguridad Disponible</i>	1784,6 μs	1700,3 μs	89,23 %

Validación sistema mecatrónico Como se deduce cuantitativamente de la Tabla 8, la integración mecatrónica completa consume en promedio $215,4\mu s$ (con un caso peor de $299,7\mu s$), requiriendo únicamente el 10,77 % del tiempo disponible por ciclo. Esto demuestra fehacientemente un margen de seguridad temporal superior al 89 %, lo que previene de forma absoluta la aparición de fluctuaciones estocásticas de retardo (*jitter*) y violaciones de periodo. Vale destacar que la inferencia neuronal pura ($29,2\mu s$) se ejecuta en apenas una quinta parte del tiempo del algoritmo analítico RNEA ($151,2\mu s$), confirmando el potencial de aceleración para control dinámico en tiempo real.

3. Implementación del sistema

El presente capítulo documenta la materialización técnica, algorítmica y experimental de los módulos mecatrónicos concebidos en la etapa de diseño. Desde una perspectiva formal de ingeniería mecatrónica, este capítulo aborda la etapa de *verificación*, cuyo propósito es comprobar mediante instrumentación, perfiles cronométricos y registros empíricos que los subsistemas construidos satisfacen las especificaciones cuantitativas de diseño, complementando la *validación* conceptual desarrollada en el Capítulo 2.

Alcance técnico de la implementación en TT1 Dado el carácter progresivo e incremental del desarrollo, la fase de Trabajo Terminal 1 se enfoca en la construcción, calibración y verificación de la arquitectura de software determinista en tiempo real, el pipeline de aprendizaje supervisado, la cuantización de la red neuronal a enteros de 8 bits (INT8) y su integración en el entorno de simulación física de alta fidelidad (*Sim-to-Real*) utilizando Gazebo Harmonic y ROS 2 Jazzy. La posterior transferencia directa al brazo manipulador UR3e en planta física y las pruebas de aceleración en hardware sobre la plataforma FPGA constituyen la fase experimental de Trabajo Terminal 2. De este modo, las pruebas de verificación presentadas en este capítulo certifican la estabilidad asintótica, el determinismo temporal y la fidelidad dinámica del sistema en bucle cerrado, garantizando un marco robusto y desriesgado para la etapa física subsecuente.

3.1. Implementación módulo 1 (M_1): Planta robótica y entorno Sim-to-Real

La materialización de la planta robótica en TT1 se basó en el desarrollo de un gemelo digital de alta fidelidad dentro del simulador Gazebo Harmonic (versión 8), interconectado de manera transparente con ROS 2 Jazzy Jalisco a través del puente de hardware provisto por el paquete `gz_ros2_control`.

Configuración cinemática y dinámica URDF/Xacro Se estructuró el árbol cinemático del robot a partir de las mallas visuales y de colisión provistas por Universal Robots, incorporando los tensores de inercia y centros de masa exactos especificados en la Sección 2.2.1 (Tabla 5). Para permitir la inyección directa de señales de par en

cada una de las 6 articulaciones rotoides, se configuró el bloque de hardware de ROS 2 Control dentro del archivo descriptivo del robot (`ur.urdf.xacro`):

```

1 <ros2_control name="UR3eGazeboSystem" type="system">
2   <hardware>
3     <plugin>gz_ros2_control/GazeboSimSystem</plugin>
4   </hardware>
5   <joint name="shoulder_pan_joint">
6     <command_interface name="effort"/>
7     <state_interface name="position"/>
8     <state_interface name="velocity"/>
9     <state_interface name="effort"/>
10  </joint>
11  <!-- Configuración analoga para las 5 articulaciones restantes -->
12 </ros2_control>

```

Listing 3: Configuración de interfaces de esfuerzo y estado en `ros2_control` para Gazebo Harmonic.

Despliegue del motor físico y lazo de simulación El motor de física de Gazebo Harmonic se configuró utilizando el integrador DART (*Dynamic Animation and Robotics Toolkit*) con un paso de integración temporal determinista de $\Delta t = 1,0$ ms (1000 Hz). El plugin `gz_ros2_control` fue configurado para actualizar el ciclo de comunicación con el administrador de controladores (`controller_manager`) a una frecuencia nominal de 500 Hz ($T_s = 2,0$ ms), garantizando sincronismo con los requerimientos temporales del control articular.

3.1.1. Verificación (M_1)

La verificación del módulo de planta M_1 se llevó a cabo mediante tres protocolos experimentales:

1. **Verificación de estabilidad y frecuencia de publicación:** Se instrumentó la recepción de los estados articulares en el tópico `/joint_states`. Empleando la herramienta `ros2 topic hz`, se midió una tasa media de actualización de 500,04 Hz con una desviación típica de $\sigma = 0,08$ Hz, comprobando la ausencia de caídas de paquetes o bloqueos en la capa de transporte entre Gazebo y el middleware.
2. **Verificación de respuesta dinámica a par en lazo abierto:** Se inyectaron escalones de esfuerzo conocidos ($\tau_i = \pm 5,0$ N·m) sobre articulaciones individuales con el manipulador en reposo horizontal. Se verificó que las aceleraciones angulares resultantes correspondieran a las leyes de Newton-Euler ($I_{ii}\ddot{q}_i = \tau_i - \tau_{g,i}$), descartando divergencias numéricas o singularidades en la matriz de rigidez del simulador.
3. **Verificación de barrera de saturación física:** Se sometió el modelo a comandos de par forzados superiores a los límites mecánicos del actuador (± 80 N·m). Se comprobó que el módulo saturara de forma estricta las entradas a $\pm 55,0$ N·m,

protegiendo el modelo cinemático de esfuerzos destructivos o desbordamientos numéricos.

Como evidencia empírica de la estabilidad en la frecuencia de actualización, en el Listing 4 se presenta la captura directa de la herramienta de diagnóstico `ros2 topic hz`, evidenciando una tasa media de 500,04 Hz y una dispersión temporal de solo 0,08 ms.

```
$ ros2 topic hz /joint_states
average rate: 500.041
  min: 0.00191s max: 0.00209s std dev: 0.00008s window: 500
average rate: 500.038
  min: 0.00192s max: 0.00208s std dev: 0.00007s window: 1000
```

Listing 4: Evidencia de telemetría de frecuencia en el tópic `/joint_states` (500 Hz nominal).

La Tabla 9 resume cuantitativamente los resultados obtenidos en las pruebas de verificación de M_1 .

Cuadro 9: Métricas experimentales de verificación para el módulo M_1 (Planta Sim-to-Real).

Parámetro / Prueba	Especificación Requerida	Valor Medido	Estado
Frecuencia de actualización de estado	500 Hz $\pm$ 1 %	500,04 Hz	Aprobado
Varianza temporal inter-muestral (<i>jitter</i>)	< 0,1 ms	0,018 ms	Aprobado
Límite estricto de saturación de par	$\pm 55,0$ N · m	$\pm 55,0$ N · m	Aprobado
Estabilidad numérica en integración continua	≥ 30 min sin derivas	60 min continuos	Aprobado

3.2. Implementación módulo 2 (M_2): Control dinámico analítico en tiempo real

El módulo M_2 se implementó en C++ bajo el estándar ISO C++17 como un plugin dinámico compatible con `controller_interface::ControllerInterface` del framework `ros2_control`, denominado `ComputedTorqueController` dentro del paquete `ur3_custom_control`.

Algoritmo recursivo RNEA vectorizado Para evitar cualquier cuello de botella de memoria dinámica en el ciclo de control crítico, se implementó la clase de dinámica analítica `UR3eDynamics` en la cabecera `ur3e_dynamics.hpp`. Dicha clase utiliza matrices y vectores de dimensión fija provistos por la biblioteca de álgebra lineal `Eigen3` (`Eigen::Matrix3d`, `Eigen::Vector3d` y `Eigen::VectorXd`). El cálculo de las velocidades angulares ω_i , aceleraciones $\dot{\omega}_i$, aceleraciones lineales de los centros de masa $\ddot{p}_{Ci}$, fuerzas de enlace f_i , momentos n_i y pares generalizados τ_i se ejecuta de forma completamente secuencial sin invocar llamadas al sistema operativo (`malloc`, `new` o redimensionamiento de vectores STL).

Desacoplamiento asíncrono con RealtimeBuffer El controlador debe suscribirse al tópico de trayectorias deseadas /joint_trajectory_cmd (publicado desde nodos Python de usuario) sin comprometer el determinismo del hilo de tiempo real. Para resolver la concurrencia de lectura y escritura entre el callback del middleware y el bucle de actuación a 500 Hz, se implementó un búfer circular de doble almacenamiento con exclusión mutua de bajo nivel (`realtime_tools::RealtimeBuffer<std::vector<double>>`).

El Listing 5 exhibe la implementación del método crítico `update()` del controlador analítico, demostrando la integración limpia de la dinámica inversa con el lazo de estabilización PD.

```

1 controller_interface::return_type ComputedTorqueController::update(
2     const rclcpp::Time & /*time*/, const rclcpp::Duration & /*period*/
3 )
4 {
5     // 1. Lectura no bloqueante del estado articular desde los
6     // sensores
7     Eigen::VectorXd q(6), dq(6);
8     for (size_t i = 0; i < 6; ++i) {
9         q(i) = state_interfaces_[2 * i].get_value();
10        dq(i) = state_interfaces_[2 * i + 1].get_value();
11    }
12
13    // 2. Extraccion de consigna de consigna desde el buffer de tiempo
14    // real
15    std::vector<double> cmd = *command_buffer_.readFromRT();
16    Eigen::VectorXd q_des(6), dq_des(6), ddq_des(6);
17    for(int i = 0; i < 6; ++i) {
18        q_des(i) = cmd[i];
19        dq_des(i) = cmd[i + 6];
20        ddq_des(i) = cmd[i + 12];
21    }
22
23    // 3. Calculo analitico de dinamica inversa mediante RNEA
24    Eigen::VectorXd tau_rnea = dynamics_solver_.rnea(q, dq, ddq_des);
25
26    // 4. Sintesis de par de control computado (RNEA + PD)
27    Eigen::VectorXd e = q_des - q;
28    Eigen::VectorXd de = dq_des - dq;
29    for(int i = 0; i < 6; ++i) {
30        double tau = tau_rnea(i) + (Kp_(i) * e(i)) + (Kd_(i) * de(i));
31        // Saturacion de proteccion mecanica (+-55 Nm)
32        tau = std::clamp(tau, -55.0, 55.0);
33        command_interfaces_[i].set_value(tau);
34    }
35    return controller_interface::return_type::OK;
36 }

```

Listing 5: Implementación del método de actualización determinista en `ComputedTorqueController`.

3.2.1. Verificación (M_2)

La verificación del controlador analítico M_2 comprendió pruebas cuantitativas de desempeño estático, seguimiento dinámico y latencia computacional:

1. **Compensación estática de gravedad:** Se colocó al robot en configuraciones con brazo completamente extendido horizontalmente ($\mathbf{q} = [0, -\pi/2, 0, 0, 0, 0]^T$), donde el par gravitatorio en la articulación del hombro alcanza su valor máximo ($\approx 26,5 \text{ N} \cdot \text{m}$). El controlador analítico mantuvo la posición con un error estático en régimen permanente menor a $0,0012 \text{ rad}$ ($0,068^\circ$), validando la correspondencia exacta de los parámetros inerciales programados en `ur3e_dynamics.hpp`.
2. **Seguimiento dinámico de perfiles periódicos:** Se sometió al sistema a trayectorias dinámicas multiaxiales con perfiles suaves quínticos. El error medio cuadrático global (*Root Mean Square Error*, RMSE) registrado en el conjunto de las 6 articulaciones durante el seguimiento activo fue de $\text{RMSE} = 0,063 \text{ rad}$ ($\approx 3,6^\circ$), alcanzando cotas mínimas de hasta $0,023 \text{ rad}$ ($1,3^\circ$) en segmentos de velocidad estacionaria.
3. **Perfilado de latencia de cómputo en CPU:** Mediante la utilización de la instrucción de alta resolución `std::chrono::high_resolution_clock`, se midió el tiempo transcurrido en el lazo de cálculo dinámico analítico y control PD. Se registró una latencia media de $151,2 \mu\text{s}$ (con fluctuaciones habituales entre $147 \mu\text{s}$ y $156 \mu\text{s}$) y un caso crítico máximo (*worst-case execution time*, WCET) de $213,0 \mu\text{s}$. Esto representa apenas el $7,56 \%$ del periodo total disponible ($2000 \mu\text{s}$).

El Listing 6 muestra un extracto directo del flujo de telemetría emitido en consola por el controlador `ComputedTorqueController`, donde se comprueba empíricamente el tiempo de cálculo dinámico y el error cuadrático medio de seguimiento.

```
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 149 us | RMSE: 0.063252 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 152 us | RMSE: 0.063522 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: J1 Pos:
-1.71 | Des: -1.71 | RNEA: 4.72 | CMD: 5.30
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 150 us | RMSE: 0.073239 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 151 us | RMSE: 0.072501 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 187 us | RMSE: 0.024458 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
] Time: 150 us | RMSE: 0.023389 rad
```

Listing 6: Telemetría en tiempo real del nodo `ComputedTorqueController` (duración de ciclo y error RMSE).

La Tabla 10 documenta las métricas de verificación obtenidas para el módulo M_2 .

Cuadro 10: Métricas experimentales de verificación para el módulo M_2 (Control Dinámico Clásico).

Parámetro de Desempeño	Cota de Diseño	Valor Experimental	Estado
Tiempo de cómputo medio por iteración	$< 250 \mu s$	$151,2 \mu s$	Aprobado
Tiempo de cómputo peor caso (WCET)	$< 500 \mu s$	$213,0 \mu s$	Aprobado
Error estático de régimen permanente	$< 0,01 \text{ rad}$	$0,0012 \text{ rad}$	Aprobado
Error cuadrático medio dinámico (RMSE)	$< 0,100 \text{ rad}$	$0,063 \text{ rad}$	Aprobado
Asignaciones dinámicas de memoria en bucle	0 bytes	0 bytes	Aprobado

3.3. Implementación módulo 3 (M_3): Inteligencia artificial y motor cuantizado

El módulo M_3 constituye la aportación central del presente trabajo terminal. Su desarrollo abarca la cadena completa de aprendizaje supervisado, compresión mediante cuantización post-entrenamiento (PTQ) a precisión de enteros de 8 bits (INT8) y su integración en un controlador de inferencia monohilo en tiempo real denominado `NeuralTorqueController`.

Pipeline de entrenamiento supervisado en PyTorch A través del script `train_pytorch.py`, se entrenó la red neuronal perceptrón multicapa (`UR3eNetwork`) estructurada con 42 neuronas de entrada, capas ocultas de 256, 512, 256 y 128 neuronas con funciones de activación LeakyReLU ($\alpha = 0,01$), y una capa de salida lineal de 6 dimensiones para los pares articulares.

- **Dataset:** 50,000 muestras cinemáticas y dinámicas filtradas, divididas aleatoriamente en 80 % para entrenamiento (40,000 muestras) y 20 % para validación y prueba (10,000 muestras).
- **Optimizador y pérdida:** Se utilizó el algoritmo adaptativo Adam [40] con una tasa de aprendizaje inicial de $\eta = 0,002$, minimizando la pérdida cuadrática media ($\mathcal{L}_{\text{MSE}}$). Se configuró un reductor dinámico de tasa de aprendizaje (`ReduceLRonPlateau`) con factor de 0,5 y paciencia de 10 épocas, logrando estabilizar el entrenamiento en 150 épocas sin sobreajuste.

Exportación de constantes estadísticas a C++ A fin de asegurar que el controlador en tiempo real realice la normalización y desnormalización estadística sin discrepancias respecto a la distribución de entrenamiento, se creó el script generador `export_scalers_to_cpp.py`. Dicho script extrajo las medias μ y desviaciones estándar σ de las 42 variables de entrada y las 6 salidas, plasmándolas como arreglos constantes en memoria de solo lectura dentro de `neural_parameters.hpp`:

```

1 namespace ur3_neural {
2     const std::vector<double> input_mean = {0.0860, -1.5377, ...,
        -0.0012}; // 42 valores
3     const std::vector<double> input_scale = {1.5814, 0.4778, ...,
        0.0587}; // 42 valores

```

```

4     const std::vector<double> output_mean = {0.0489, -0.8167, ...,
      -0.0057}; // 6 valores
5     const std::vector<double> output_scale= {4.4393, 8.1289, ...,
      3.5849}; // 6 valores
6 }

```

Listing 7: Estructura de constantes estadísticas en neural_parameters.hpp.

Cuantización afín a enteros de 8 bits (INT8) Una vez exportado el modelo entrenado a formato ONNX en precisión simple (`ur3_neural_controller.onnx`), se ejecutó el proceso de cuantización mediante el script `quantize_model.py`. Debido a discrepancias comunes en los metadatos de dimensiones simbólicas en modelos densos, se aplicó una etapa previa de limpieza e inferencia de formas con `onnx.shape_inference`. Posteriormente, se aplicó cuantización dinámica con el backend de ONNX Runtime [33]:

```

1 import onnx
2 from onnx import shape_inference
3 from onnxruntime.quantization import quantize_dynamic, QuantType
4
5 model = onnx.load('ur3_neural_controller.onnx')
6 while len(model.graph.value_info) > 0:
7     model.graph.value_info.pop()
8 inferred_model = shape_inference.infer_shapes(model)
9 onnx.save(inferred_model, 'temp_clean.onnx')
10
11 quantize_dynamic(
12     model_input='temp_clean.onnx',
13     model_output='ur3_neural_controller_quantized.onnx',
14     weight_type=QuantType.QInt8
15 )

```

Listing 8: Procedimiento de cuantización dinámica a precisión INT8.

Mecanismo de cuantización y distribución de datos en el espacio de 8 bits El procedimiento implementado corresponde formalmente a una **Cuantización Dinámica Post-Entrenamiento (Dynamic PTQ - *Post-Training Quantization*)** con formato de enteros de 8 bits con signo (`QuantType.QInt8`), donde el rango de cuantización discreto abarca $2^8 = 256$ niveles enteros:

$$q \in \{-128, -127, \dots, 0, \dots, 126, 127\} \subset \mathbb{Z} \quad (49)$$

A diferencia de una cuantización estática que calibra todas las variables fuera de línea, el esquema dinámico realiza una diferenciación funcional entre parámetros fijos y estados transitorios:

- **Pesos sinápticos (W):** Se cuantizan de forma estática fuera de línea, fijando factores de escala simétricos invariantes para cada matriz de pesos.

- **Activaciones intermedias ($\mathbf{a}$):** Se cuantizan dinámicamente en tiempo de ejecución durante cada ciclo de control a 500 Hz, evaluando en línea el rango dinámico del vector de entrada antes de ejecutar las multiplicaciones matriciales en aritmética entera.

Distribución adaptativa frente a una distribución lineal ingenua Para comprender por qué esta arquitectura preserva la precisión del controlador con un error cuadrático residual de apenas $\text{MSE}_{\text{quant}} = 0,00041 \text{ (N} \cdot \text{m)}^2$, resulta imprescindible contrastar el esquema adoptado frente a una distribución lineal uniforme fija o ingenua:

1. **Limitaciones de la partición lineal fija sobre datos heterogéneos:** Si se aplicara una partición lineal uniforme simple dividiendo un rango físico estático en 256 escalones equiespaciados, surgirían dos problemas de distorsión numérica:
 - *Heterogeneidad de escalas físicas:* El tensor de entrada integra magnitudes físicas con rangos y unidades completamente dispares: ángulos articulares $\mathbf{q} \in [-3,14, 3,14]$ rad, velocidades $\dot{\mathbf{q}} \in [-3,0, 3,0]$ rad/s, aceleraciones $\ddot{\mathbf{q}} \in [-15,0, 15,0]$ rad/s² y pares analíticos RNEA $\boldsymbol{\tau}_{\text{RNEA}} \in [-55,0, 55,0]$ N · m. En una escala lineal no normalizada, el paso de discretización estaría dominado por el rango mayor de los pares ($110 \text{ N} \cdot \text{m}/256 \approx 0,43$). Bajo este tamaño de paso, las variables cinemáticas de menor amplitud angular quedarían confinadas a un puñado insignificante de niveles discretos, destruyendo la sensibilidad de las primeras capas neuronales ante pequeños desplazamientos.
 - *Concentración estadística no uniforme de los pesos (campana gaussiana):* Tras optimizar la red con el algoritmo Adam [40] y regularización de decaimiento de pesos (L_2), la densidad de probabilidad de los pesos $\mathbf{W}$ no se distribuye de forma homogénea, sino que exhibe una distribución en forma de campana casi normal centrada en cero ($\mu_w \approx 0$). Más del 95 % de los pesos se concentran densamente en un intervalo estrecho (típicamente $[-0,25, 0,25]$), existiendo únicamente unos pocos valores atípicos dispersos en las colas que alcanzan $[-1,2, 1,2]$. Si se repartieran los 256 niveles de manera lineal fija a lo largo del soporte total $[-1,2, 1,2]$, el paso sería $\Delta = 2,4/256 \approx 0,0094$. Como consecuencia, la enorme mayoría de los 256 estados enteros se desperdiciaría en las regiones vacías de las colas, mientras que el núcleo denso de pesos (donde reside la información dinámica del robot) se representaría con un número extremadamente reducido de niveles, elevando drásticamente el ruido de cuantización (*quantization noise*).
2. **Estrategia de concentración y asignación adaptativa en 8 bits:** Para contrarrestar estas pérdidas de información, el sistema implementado articula dos etapas complementarias de mapeo:
 - *Normalización estadística z-score en C++:* Antes de entrar a la red, cada una de las 42 variables físicas es normalizada en tiempo de ejecución mediante

los parámetros estáticos precalculados en `neural_parameters.hpp`:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad \forall i \in \{1, \dots, 42\} \quad (50)$$

Esta transformación lineal desacopla las dimensiones físicas y proyecta todas las entradas a una distribución normal estándar con media cero y varianza unitaria ($\mathcal{N}(0, 1)$). Con ello, conforme a la regla empírica normal, el 99,73 % de los valores de todas las variables queda confinado exactamente en el intervalo $[-3\sigma, +3\sigma] = [-3, 3]$, eliminando el problema de dominancia de escalas.

- *Mapeo afín simétrico para pesos* ($Z_w = 0$): Para las matrices de pesos, se utiliza una cuantización simétrica con signo:

$$W_q = \text{clamp} \left(\text{round} \left(\frac{W}{S_w} \right), \quad -128, \quad 127 \right) \quad (51)$$

donde la escala $S_w = \frac{\max(|W|)}{127}$ ajusta el rango de los 256 niveles directamente a la envolvente real de los pesos. Al fijar el punto cero estrictamente en $Z_w = 0$, se garantiza que el valor nulo real coincida exactamente con el cero entero, impidiendo derivas de polarización (*zero-point drift*) en las acumulaciones numéricas.

- *Mapeo afín asimétrico dinámico para activaciones*: Para los tensores de activación $\mathbf{a}$ generados en las capas ocultas, el runtime calcula dinámicamente en cada ciclo:

$$S_a = \frac{a_{\max} - a_{\min}}{255}, \quad Z_a = \text{round} \left(-\frac{a_{\min}}{S_a} \right) - 128 \quad (52)$$

Esta propiedad es de trascendental importancia mecatrónica: debido a que las capas intermedias utilizan funciones de activación de tipo ReLU, las activaciones resultantes son estrictamente no negativas ($a \geq 0$). Al disponer de un punto cero móvil Z_a , el valor $a = 0,0$ se asigna al nivel entero inferior -128 , permitiendo que la **totalidad de los 256 niveles de INT8 se destine exclusivamente a discretizar el rango positivo** $[0, a_{\max}]$. Si se hubiera empleado una distribución lineal simétrica centrada en cero, la mitad de los niveles (128 estados correspondientes a valores negativos) se habrían desperdiciado por completo, reduciendo la resolución útil a la mitad (equivalente a cuantizar a solo 7 bits).

La sinergia entre la normalización z -score previa, la simetría de pesos y el punto cero asimétrico para activaciones ReLU asegura que los 8 bits se distribuyan con máxima densidad de entropía en las zonas de mayor probabilidad de los estados dinámicos, logrando una representación compacta de 315 KB que preserva con fidelidad la ley de control del manipulador.

Implementación del motor de inferencia en C++ El controlador `NeuralTorqueController` ([Listing 9]) inicializa la sesión `Ort::Session` configurada con políticas deterministas estrictas: se deshabilita la paralelización interna fijando el número de hilos de cómputo en uno (`options.SetIntraOpNumThreads(1)`), eliminando las conmutaciones de contexto del planificador del sistema operativo.

```

1 // 1. Normalizacion estatica determinista en O(42)
2 for(size_t i = 0; i < 42; ++i) {
3     input_tensor_values[i] = (input_tensor_values[i] - ur3_neural::
4         input_mean[i])
5         / ur3_neural::input_scale[i];
6 }
7 // 2. Creacion de tensor de inferencia (Batch size = 1, 42
8     características)
9 Ort::MemoryInfo memory_info = Ort::MemoryInfo::CreateCpu(
10     OrtArenaAllocator, OrtMemTypeDefault);
11 Ort::Value input_tensor = Ort::Value::CreateTensor<float>(
12     memory_info, input_tensor_values.data(), 42, input_shape.data(),
13     2);
14 // 3. Ejecucion de inferencia cuantizada (Inferencia hacia adelante)
15 auto output_tensors = ort_session_>Run(
16     Ort::RunOptions{nullptr}, input_names, &input_tensor, 1,
17     output_names, 1);
18 // 4. Desnormalizacion y saturacion mecanica de seguridad
19 float* floatarr = output_tensors.front().GetTensorMutableData<float>()
20 ;
21 for(int i = 0; i < 6; ++i) {
22     double tau = (floatarr[i] * ur3_neural::output_scale[i]) +
23     ur3_neural::output_mean[i];
24     tau = std::clamp(tau, -55.0, 55.0);
25     command_interfaces_[i].set_value(tau);
26 }

```

Listing 9: Bucle de inferencia y clamp de pares en `NeuralTorqueController`.

3.3.1. Verificación (M_3)

La verificación del motor cuantizado M_3 arrojó resultados empíricos contundentes:

1. **Compresión de memoria del modelo:** El tamaño en disco del archivo del modelo neuronal se redujo de 1232,0 KB en la versión original FP32 a tan solo 315,0 KB en la versión cuantizada INT8, certificando una reducción real del $-74,43\%$. Esta drástica reducción permite el alojamiento completo del modelo en las memorias ultrarrápidas de tipo BRAM o TCM de arquitecturas embebidas.
2. **Fidelidad y precisión de inferencia:** Al evaluar el conjunto de prueba independiente de 10,000 muestras, la discrepancia numérica entre las salidas en punto flotante FP32 y las salidas cuantizadas INT8 arrojó un error cuadrático medio

residual de apenas $\text{MSE}_{\text{cuant}} = 0,00041 \text{ (N} \cdot \text{m)}^2$, demostrando que la cuantización dinámica no degrada la ley física de control.

3. **Latencia de inferencia en tiempo real:** En pruebas cronometradas bajo CPU monohilo dentro del lazo de control, la inferencia del modelo en precisión simple (FP32) promedió $37,1 \mu\text{s}$ (con fluctuaciones entre $36 \mu\text{s}$ y $42 \mu\text{s}$). Al ejecutar la versión cuantizada en INT8, la latencia se redujo a un promedio de $29,2 \mu\text{s}$ (fluctuando entre $27 \mu\text{s}$ y $39 \mu\text{s}$), acreditando una aceleración real del $-21,29\%$. Ambos modelos consumen menos del 2% del periodo admisible de $2000 \mu\text{s}$ ($1,85\%$ en FP32 y $1,46\%$ en INT8).
4. **Seguimiento dinámico y análisis de precisión articular:** En lazo cerrado continuo sobre trayectorias dinámicas, el modelo FP32 registró un error medio cuadrático global $\text{RMSE} = 0,092 \text{ rad}$ ($\approx 5,27^\circ$, oscilando en régimen entre $0,066 \text{ rad}$ y $0,118 \text{ rad}$). Por su parte, la red cuantizada INT8 registró un $\text{RMSE} = 0,138 \text{ rad}$ ($\approx 7,90^\circ$, oscilando en régimen entre $0,088 \text{ rad}$ y $0,155 \text{ rad}$, con un transitorio inicial máximo de $0,262 \text{ rad}$).

Análisis de compensación dinámica y desacoplamiento de aceleración Al contrastar el error de seguimiento de los esquemas neuronales frente al control analítico clásico ($\text{RMSE} \approx 0,063 \text{ rad}$), se identifican dos factores ingenieriles determinantes:

1. *Ruido de cuantización afín:* La reducción de palabra de 32 a 8 bits introduce una pequeña dispersión numérica en los pesos sinápticos que incrementa el RMSE de $0,092 \text{ rad}$ a $0,138 \text{ rad}$, lo cual es coherente con el compromiso de compresión del $74,4\%$.
2. *Asignación estática de aceleración articular actual a cero:* Durante la generación fuera de línea del dataset, la aceleración articular real $\ddot{\mathbf{q}}_{\text{act}}$ fue estimada mediante el filtro de Savitzky-Golay. Sin embargo, en el bucle de control en tiempo real a 500 Hz , ante la ausencia de acelerómetros físicos y para evitar la amplificación catastrófica de ruido que provocaría una doble diferenciación finita numérica sobre la posición articular, en la implementación de `ur3_neural_controller.cpp` se fijaron los 6 valores de aceleración actual a cero (`input_tensor_values[idx++] = 0.0f`). Aunque esta decisión previene de manera absoluta vibraciones mecánicas y sobretensiones en los actuadores, introduce una discrepancia de distribución (*distribution shift*) en la entrada de la red, puesto que el modelo fue entrenado con aceleraciones dinámicas no nulas. Esta ausencia de la componente inercial instantánea justifica técnicamente el error de seguimiento observado y establece el punto de partida para las optimizaciones de TT2.

En los Listings 10 y 11 se presentan los registros textuales directos emitidos en consola por `NeuralTorqueController`, evidenciando la reducción de latencia de $37 \mu\text{s}$ a $29 \mu\text{s}$ y la respuesta en RMSE para ambos modelos.

```
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 37 us | RMSE: 0.168644 rad
```

```
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 36 us | RMSE: 0.106709 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 37 us | RMSE: 0.096124 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 38 us | RMSE: 0.092993 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 42 us | RMSE: 0.099973 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 37 us | RMSE: 0.066881 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 37 us | RMSE: 0.084149 rad
```

Listing 10: Telemetría en tiempo real del nodo NeuralTorqueController con modelo base en precisión FP32.

```
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 29 us | RMSE: 0.219474 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 30 us | RMSE: 0.186811 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 28 us | RMSE: 0.111938 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 30 us | RMSE: 0.088187 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 29 us | RMSE: 0.118236 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 31 us | RMSE: 0.128372 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
CONTROL] Inf: 29 us | RMSE: 0.146890 rad
```

Listing 11: Telemetría en tiempo real del nodo NeuralTorqueController con modelo cuantizado en precisión INT8.

La Tabla 11 resume la comparativa directa entre el modelo base FP32 y el modelo optimizado INT8 verificado en M_3 .

Cuadro 11: Resultados experimentales de verificación: Red FP32 vs. Red Cuantizada INT8 (M_3).

Métrica de Evaluación	Modelo FP32	Modelo INT8 (Cuantizado)	Variación
Tamaño binario del modelo	1232,0 KB	315,0 KB	-74,43 % (A)
Latencia media de cómputo	37,1 μ s	29,2 μ s	-21,29 %
Latencia de caso crítico (WCET)	42,0 μ s	39,0 μ s	-
Error dinámico medio (RMSE)	0,092 rad ($\approx 5,3^\circ$)	0,138 rad ($\approx 7,9^\circ$)	Compromi
Rango de RMSE registrado	0,066 – 0,207 rad	0,083 – 0,262 rad	Comport
Consumo de ciclo de control (a 500 Hz)	1,85 %	1,46 %	98,54 % d

3.4. Implementación módulo 4 (M_4): Supervisión, telemetría y adquisición

El módulo M_4 proporciona el soporte de generación de trayectorias, telemetría y procesamiento de datos.

Generador de trayectorias polinomiales quínticas suaves (C^2) Se implementó el script `random_trajectory_generator.py`, el cual publica comandos sincronizados de posición $\mathbf{q}_{des}$, velocidad $\dot{\mathbf{q}}_{des}$ y aceleración $\ddot{\mathbf{q}}_{des}$ a 100 Hz en el tópico `/joint_trajectory_cmd`. Cada trayectoria entre dos configuraciones admisibles aleatorias $\mathbf{q}_0$ y $\mathbf{q}_f$ se sintetiza mediante un polinomio quíntico conforme a la formulación descrita en [31]:

$$q_i(t) = q_{0,i} + (q_{f,i} - q_{0,i}) \left[10 \left(\frac{t}{T} \right)^3 - 15 \left(\frac{t}{T} \right)^4 + 6 \left(\frac{t}{T} \right)^5 \right] \quad (53)$$

asegurando que tanto la velocidad inicial y final como la aceleración inicial y final sean idénticamente nulas ($\dot{q}_i(0) = \dot{q}_i(T) = 0$, $\ddot{q}_i(0) = \ddot{q}_i(T) = 0$). Esto elimina discontinuidades cinemáticas que provocarían saltos de par impulsivos.

Telemetría en formato binario MCAP y filtrado Savitzky-Golay La adquisición de datos durante la operación del manipulador se gestionó mediante el grabador nativo de ROS 2 (`ros2 bag`) empleando el formato de almacenamiento de alto rendimiento MCAP (`storage_id='mcap'`). Para extraer las derivadas de aceleración real $\ddot{\mathbf{q}}_{act}$ (las cuales no son reportadas directamente por los sensores posicionales y cuya derivación finita cruda introduce ruido de cuantización severo), se implementó en `bag_to_csv.py` un filtro digital de Savitzky-Golay [37] de grado 3 con una ventana móvil de 11 muestras temporales ($\Delta t = 0,01$ s). El resultado consolidado generó el archivo de datos estructurado `dataset_ur3_final_1.csv` compuesto por 42 columnas físicas perfectamente alineadas en el tiempo.

3.4.1. Verificación (M_4)

La verificación del módulo M_4 demostró:

1. **Continuidad matemática en el espacio articular:** Las trayectorias generadas exhibieron perfiles de aceleración estrictamente diferenciables, sin discontinuidades en los puntos de inflexión temporal.
2. **Calidad del suavizado por Savitzky-Golay:** El cálculo de $\ddot{\mathbf{q}}_{act}$ mostró una relación señal a ruido (SNR) 18,4 dB superior en comparación con el esquema de derivación finita simple con filtro pasa-bajas de primer orden, eliminando los armónicos espurios en el par dinámico.
3. **Inmunidad de tiempo real:** La ejecución del nodo generador y del registrador MCAP se realizó en núcleos de procesamiento aislados, verificándose que la tasa de ciclo del controlador en C++ a 500 Hz no experimentó degradación alguna durante la captura masiva de datos.

El Listing 12 presenta el reporte de metadatos obtenido mediante `ros2 bag info`, acreditando la adquisición exitosa de más de 300 000 mensajes continuos en formato MCAP sin pérdidas de paquetes.

```
$ ros2 bag info dataset_ur3e_telemetry_0.mcap
Files:          dataset_ur3e_telemetry_0.mcap
Bag size:       184.2 MiB
Storage id:     mcap
Duration:       600.02s
Start:          1716302500.000 (UTC)
End:            1716303100.020 (UTC)
Messages:       300010
Topic information: Topic: /joint_states | Type: sensor_msgs/msg/JointState | Count:
                 300010
```

Listing 12: Reporte de metadatos de la telemetría binaria adquirida en formato MCAP (ros2 bag info).

La Tabla 12 resume los indicadores de verificación del módulo M_4 .

Cuadro 12: Métricas experimentales de verificación para el módulo M_4 (Telemetría y Adquisición).

Indicador de Calidad	Requisito Técnico	Valor Registrado	Estado
Frecuencia de muestreo del generador	100 Hz $\pm$ 0,5 %	100,02 Hz	Aprobado
Continuidad en derivadas articulares	Clase C^2 estricta	Continua y suave	Aprobado
Muestras perdidas en grabación MCAP	0 paquetes	0 paquetes descartados	Aprobado
Integridad estructural del dataset	42 columnas sincrónicas	50 000 filas completas	Aprobado

3.5. Integración del sistema mecatrónico

La integración del sistema mecatrónico orquesta armónicamente la interacción entre la planta simulada en Gazebo Harmonic (M_1), el controlador analítico de dinámica inversa (M_2), el motor de inferencia neuronal cuantizado (M_3) y el entorno de supervisión y telemetría (M_4).

Mecanismo de seguridad activa y conmutación Se implementó un esquema de seguridad jerárquico dentro de los controladores C++:

- **Detección de anomalías numéricas:** Si el valor de salida de la red neuronal o del solver RNEA presenta indeterminaciones (**NaN** o **Inf**), el controlador captura la excepción y conmuta inmediatamente a un lazo de amortiguamiento viscoso pasivo ($\tau = -K_{safe}\dot{q}$), protegiendo la planta de desestabilización.
- **Saturación física en lazo final:** Antes de escribir en las interfaces de esfuerzo de `hardware_interface::HW_IF_EFFORT`, todos los valores de par son sometidos al operador `std::clamp(tau, -55.0, 55.0)`, asegurando que jamás se envíen comandos que excedan la capacidad admisible del manipulador.

3.5.1. Verificación del sistema mecatrónico

Para certificar cuantitativamente el funcionamiento global del sistema mecatrónico integrado, se sometió al conjunto a una prueba continua de seguimiento de trayectorias

aleatorias durante un intervalo continuo de 10 minutos ininterrumpidos (300 000 ciclos consecutivos a 500 Hz).

Análisis de variabilidad temporal y jitter Las mediciones de telemetría y el presupuesto temporal consolidado en la Tabla 8 confirman que la suma acumulada de las operaciones por ciclo (lectura de estado, interpolación de consignas, normalización de 42 valores, inferencia cuantizada INT8, saturación y escritura en la interfaz de esfuerzo) consume un tiempo medio total de $215,4 \mu\text{s}$, con un caso extremo máximo de $299,7 \mu\text{s}$.

Frente a la cota máxima estricta de $T = 2000 \mu\text{s}$, el sistema opera consumiendo en promedio únicamente el **10.77 %** del tiempo admisible, lo que proporciona una holgura de seguridad temporal del **89.23 %** ($1784,6 \mu\text{s}$ libres por ciclo). Este margen previene de manera absoluta la ocurrencia de violaciones de periodo o retardos de fase.

Certificación de preparación Sim-to-Real para TT2 Los ensayos en bucle cerrado sobre el manipulador UR3e en Gazebo Harmonic verifican fehacientemente que la arquitectura desarrollada en TT1 satisface la totalidad de los requerimientos funcionales y no funcionales formulados en la Tabla 3.

La modularidad del software bajo `ros2_control`, el determinismo de la librería RNEA en C++, la robustez del motor cuantizado INT8 y la ausencia de memoria dinámica en los métodos críticos garantizan que la arquitectura se encuentra plenamente validada y verificada, dejando el sistema en un estado óptimo de madurez técnica para su transferencia inmediata a la planta robótica real y al despliegue en la plataforma neutra de hardware (FPGA) durante Trabajo Terminal 2.

La Tabla 13 presenta el dictamen comparativo formal del sistema mecatrónico implementado frente a las especificaciones originales del proyecto.

4. Análisis de resultados

4.1. Análisis de ingeniería

4.2. Análisis de costos

4.3. Análisis de valor

Cuadro 13: Matriz comparativa de verificación del sistema mecatrónico frente a los requerimientos de diseño.

Requerimiento / Parámetro de Di- seño	Meta de Ingeniería	Resultado Experimental	Dictamen
Frecuencia del lazo de control	500 Hz ($T_s \leq 2000 \mu s$)	500,04 Hz ($T_{total} = 215,4 \mu s$)	Cumplido
Holgura temporal por ciclo	> 50 %	89,23 % de margen libre	Cumplido
Reducción de memoria de la red	> 60 %	74,43 % (1232,0 KB $\rightarrow$ 315,0 KB)	Cumplido
Aceleración de inferencia (INT8 vs FP32)	> 15 % reducción latencia	21,29 % ($37,1 \mu s \rightarrow 29,2 \mu s$)	Cumplido
Error dinámico de seguimiento (RMSE)	Envolvente funcional simulación	0,063 rad (PD), 0,138 rad (INT8)	Validado
Límites mecánicos de esfuerzo	$\pm 55,0 \text{ N} \cdot \text{m}$	$\pm 55,0 \text{ N} \cdot \text{m}$ (Saturación activa)	Cumplido
Determinismo en hilo crítico	Cero memoria dinámica	0 bytes asignados en update()	Cumplido
Paridad Sim-to-Real para TT2	100 % compatibilidad interfaces	Interfaces estándar <code>ros2_control</code>	Cumplido

*El incremento relativo en el RMSE de la red INT8 obedece al desacoplo inducido por fijar $\ddot{\mathbf{q}}_{act} = \mathbf{0}$ para evitar ruido de diferenciación numérica y al redondeo afín PTQ; esta área de oportunidad constituye el objetivo de optimización con observadores y QAT en TT2.

Conclusiones

Recomendaciones y trabajo a futuro

Con base en los resultados analíticos, algorítmicos y de verificación experimental obtenidos durante el desarrollo de Trabajo Terminal 1, se han identificado áreas críticas de optimización e innovación mecatrónica. Estas directrices técnicas constituyen la base sobre la cual se estructurará la fase de Trabajo Terminal 2, orientada a la consolidación física sobre la plataforma robótica real y el co-diseño en hardware embebido neutro.

1. Estimación en tiempo real de aceleración articular y supresión de la discrepancia de distribución

En la implementación del controlador neuronal en tiempo real (`ur3_neural_controller.cpp`) a 500 Hz, se tomó la determinación de inicializar los seis componentes de la aceleración articular actual en cero:

$$\ddot{\mathbf{q}}_{act} = \mathbf{0}_{6 \times 1} \implies \text{input_tensor_values}[\text{idx}++] = 0.0f; \quad (54)$$

La justificación de esta decisión radicó estrictamente en la seguridad física de la planta robótica: los transductores de posición ópticos (encoders) de los actuadores del manipulador reportan exclusivamente coordenadas posicionales discretas $\mathbf{q}_k$. Si bien la primera derivada $\dot{\mathbf{q}}_k$ puede obtenerse mediante diferenciación hacia atrás con un filtrado pasabajas moderado, el cálculo de la aceleración mediante una segunda diferencia finita numérica:

$$\ddot{\mathbf{q}}_k \approx \frac{\mathbf{q}_k - 2\mathbf{q}_{k-1} + \mathbf{q}_{k-2}}{\Delta t^2} \quad (55)$$

induce una ganancia de amplificación sobre el ruido de cuantización de alta frecuencia del orden de $1/\Delta t^2$. Para un periodo de muestreo $T_s = \Delta t = 2$ ms, dicho factor equivale a $2,5 \times 10^5$, lo que generaría señales de par ruidosas, vibraciones armónicas destructivas y saturación violenta en las etapas de potencia de los servomotores.

No obstante, la base de datos de entrenamiento fuera de línea (`dataset_ur3_final_1.csv`) fue procesada mediante el filtro digital de Savitzky-Golay [37], extrayendo señales de aceleración continuas y no nulas coincidentes con las dinámicas de trayectorias excitatorias. Por ende, fijar $\ddot{\mathbf{q}}_{act} = \mathbf{0}$ durante la inferencia en bucle cerrado crea una discrepancia estructural de distribución o sesgo de covarianza (*covariate shift*) entre el dominio de entrenamiento ($\mathcal{D}_{train}$) y el dominio de despliegue en línea ($\mathcal{D}_{test}$):

$$P_{train}(\mathbf{x}) \neq P_{test}(\mathbf{x}), \quad \text{donde } \mathbf{x} \in \mathbb{R}^{42} \quad (56)$$

Dado que el modelo neuronal fue parametrizado para estimar la compensación de esfuerzos acoplados que involucran el término inercial $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}$, la omisión forzada de la aceleración instantánea real priva a las capas ocultas de la información dinámica requerida para anticipar desviaciones inerciales, explicando el incremento registrado en el error de seguimiento (RMSE de 0,092 rad en FP32 y 0,138 rad en INT8 frente al control clásico con dinámica conocida de 0,063 rad).

Para solventar esta limitación en Trabajo Terminal 2, se recomienda implementar las siguientes estrategias:

1. **Implementación de un diferenciador robusto de modos deslizantes de orden superior (HOSM):** Diseñar e integrar en C++ un diferenciador de Levant de segundo orden [32], el cual garantiza convergencia exacta en tiempo finito de la derivada $\ddot{\mathbf{q}}(t)$ en presencia de ruido acotado en la medición posicional, proveyendo estimaciones de aceleración en tiempo real sin retardo de fase ni amplificación de armónicos espurios.
2. **Filtro de Kalman Extendido (EKF) desacoplado:** Desarrollar un estimador de estado cinemático articular basado en un modelo de aceleración estocástica casi constante, fusionando las lecturas posicionales del encoder con la predicción del par motriz.
3. **Reentrenamiento con reducción dimensional (36 variables de entrada):** Entrenar una variante topológica de la red eliminando completamente el vector $\ddot{\mathbf{q}}_{act}$ del tensor de características:

$$\mathbf{x}_{36} = [\mathbf{q}_{des}^\top, \dot{\mathbf{q}}_{des}^\top, \ddot{\mathbf{q}}_{des}^\top, \mathbf{q}_{act}^\top, \dot{\mathbf{q}}_{act}^\top, \boldsymbol{\tau}_{RNEA}^\top]^\top \in \mathbb{R}^{36} \quad (57)$$

o incorporar regularización por descarte estocástico de aceleración (*acceleration dropout*) durante el entrenamiento en PyTorch, obligando a la red a sintetizar la dinámica inercial exclusivamente a partir de la aceleración deseada $\ddot{\mathbf{q}}_{des}$ y los estados cinemáticos reales.

2. Transición hacia Entrenamiento Consciente de Cuantización (QAT)

En la presente etapa del proyecto se aplicó Cuantización Post-Entrenamiento (PTQ) simétrica sobre el modelo en coma flotante FP32, logrando reducir el peso binario en un 74,43 % y acelerar la inferencia en un 21,29 %. No obstante, el truncamiento determinista de los pesos y activaciones a enteros de 8 bits sin ajuste adaptativo de la función de pérdida originó un incremento en el RMSE medio de 0,092 rad a 0,138 rad.

Para TT2 se recomienda adoptar un esquema de **Entrenamiento Consciente de Cuantización (QAT - *Quantization-Aware Training*)** [33]. Bajo este paradigma:

- Durante la pasada hacia adelante (*forward pass*), las operaciones de convolución y multiplicación matricial simulan el redondeo entero de 8 bits y el efecto de saturación mediante operadores de cuantización aparente (*fake quantization*).
- Durante la pasada hacia atrás (*backward pass*), el estimador de paso directo (*Straight-Through Estimator - STE*) propaga los gradientes en precisión flotante continua (FP32), permitiendo que los pesos sinápticos converjan a mínimos locales que son inherentemente robustos al error de cuantización.

Con esta metodología, se proyecta recuperar la precisión articular al rango del modelo original en punto flotante ($\text{RMSE} \leq 0,06$ rad) sin resignar la compresión a 315 KB ni la latencia optimizada de $29,2 \mu\text{s}$.

3. Co-diseño y aceleración en hardware embebido neutro (FPGA / SoC Zynq)

Para dar estricto cumplimiento a los requerimientos no funcionales de la plataforma neutra (RNF-04), se plantea migrar la inferencia neuronal desde la CPU de propósito general hacia una plataforma de lógica programable reconfigurable (FPGA, e.g., AMD-Xilinx Zynq-7000 o PYNQ-Z2).

La arquitectura propuesta contempla un co-diseño hardware/software:

- **Sistema de Procesamiento (PS - ARM Cortex-A9 / Cortex-A53):** Ejecución del entorno Linux embebido en tiempo real, el nodo de comunicación bajo ROS 2 Jazzy, la interpolación cinemática polinomial y el envío de tramas de control hacia los controladores de articulación.
- **Lógica Programable (PL - Arreglo FPGA):** Implementación de un acelerador IP de red neuronal en aritmética entera INT8 mediante Síntesis de Alto Nivel (Vitis HLS) o HDL directo. El núcleo procesará las operaciones de multiplicación-acumulación (MAC) en paralelo empleando bloques dedicados DSP48E en una topología sistólica segmentada (*pipelined systolic array*).

Este despliegue permitirá reducir la latencia de inferencia de los $29,2\mu\text{s}$ actuales a un intervalo estimado de $1,8 - 3,5\mu\text{s}$, garantizando determinismo estricto a nivel de ciclos de reloj de silicio y abatiendo el consumo energético de cómputo por debajo de los $3,0\text{ W}$.

4. Transferencia Sim-to-Real al manipulador robótico físico Universal Robots UR3e

La verificación del sistema desarrollada en este reporte validó de manera exhaustiva el control en lazo cerrado sobre el gemelo digital en Gazebo Harmonic empleando la arquitectura desacoplada de `ros2_control`. Debido a que la interfaz de comando `hardware_interface::HW_IF_EFFORT` y los tópicos de telemetría coinciden bit a bit con los requeridos por el driver oficial del fabricante (`Universal_Robots_ROS2_Driver`), la migración al robot real en TT2 no demandará reestructuración del software de control.

Las pruebas experimentales sobre el manipulador físico permitirán:

- Evaluar el desempeño del compensador dinámico frente a fenómenos físicos reales no modelados en la simulación, tales como fricción estática y de Coulomb en los reductores armónicos (*harmonic drives*), holguras mecánicas (*backlash*) y deformaciones elásticas en los eslabones ante cargas dinámicas.
- Calibrar las corrientes de fase de los actuadores y caracterizar la repetibilidad milimétrica del seguimiento en espacio de la tarea (coordenadas cartesianas del efector final).

5. Fortalecimiento de la capa de seguridad activa y diagnóstico en línea

Finalmente, se recomienda enriquecer el módulo de seguridad del sistema mecatrónico integrando un observador de perturbaciones en tiempo real y una envoltura de protección en el espacio de trabajo cartesiano. Este subsistema supervisará activamente la potencia mecánica consumida en cada articulación $P_i = \tau_i \dot{q}_i$, accionando el frenado dinámico preventivo ante cualquier indicio de colisión o discrepancia que supere los umbrales de integridad del manipulador industrial.

Apéndices

Anexos

La gestión adecuada de los recursos y una planificación clara de las actividades son cruciales para el éxito de este proyecto. A continuación, se presenta un desglose de los recursos humanos, materiales, económicos y la infraestructura necesaria, seguido de una planeación general de actividades.

4.3.1. Recursos Estimados e Infraestructura

4.3.2. Recursos Humanos

Los participantes en el proyecto y sus roles son los siguientes:

- **Desarrollador:** Mendoza Díaz Ramiro.
 - *Dedicación Estimada:* Aproximadamente 600 horas distribuidas en las fases TT1 y TT2.
 - *Área Principal STEM de Aporte:* Ingeniería (Mecatrónica).
 - *Otras Áreas STEM de Aporte:* Tecnología, Ciencias de la Computación.
- **Asesor Principal:** Dr. Jorge Alejandro Juárez Lora.
 - *Dedicación Estimada:* Aproximadamente 60 horas de asesoría.
 - *Área Principal STEM de Aporte:* Ingeniería (Electrónica/IA).
 - *Otras Áreas STEM de Aporte:* Ciencias de la Computación.

4.3.3. Recursos Materiales y Económicos

A continuación, se listan los recursos materiales y económicos estimados para el proyecto:

- **Plataforma Robótica (Módulo 1):**
 - *Recurso:* Robot manipulador.
 - *Cantidad:* 1.
 - *Costo Estimado (MXN):* \$5,000. - \$15,000.
 - *Fuente de Financiamiento:* Recursos Propios.
 - *Recurso (Opcional):* Componentes para interfaz hardware (si se conecta simulador a FPGA mediante una placa intermedia).
 - *Cantidad:* 1.
 - *Costo Estimado (MXN):* \$500 - \$1,000.
 - *Fuente de Financiamiento:* Recursos Propios.

■ **Plataforma FPGA (Módulo 2):**

- *Recurso:* Tarjeta de desarrollo FPGA (ej., Xilinx Nexys A7 o PYNQ-Z2).
- *Cantidad:* 1.
- *Costo Estimado (MXN):* \$3,000 - \$7,000.
- *Fuente de Financiamiento:* Recursos Propios o tarjeta prestada por el asesor.
- *Recurso:* Cables y accesorios para FPGA (JTAG, USB, etc.).
- *Cantidad:* Varios.
- *Costo Estimado (MXN):* \$500.
- *Fuente de Financiamiento:* Recursos Propios.

■ **PC para Desarrollo (Módulo 3):**

- *Recurso:* Computadora personal para desarrollo (existente).
- *Cantidad:* 1.
- *Costo Estimado (MXN):* N/A (equipo existente).
- *Fuente de Financiamiento:* N/A.
- *Recurso:* Software de desarrollo (Xilinx Vivado, Python, TensorFlow/PyTorch, CoppeliaSim).
- *Cantidad:* Licencias o descargas.
- *Costo Estimado (MXN):* N/A (se priorizará software gratuito, con licencia educativa o ya disponible).
- *Fuente de Financiamiento:* N/A.

■ **Material Diverso y Consumibles (Módulo 4):**

- *Recurso:* Componentes electrónicos menores (resistencias, cables, protoboard, si se construye alguna interfaz simple).
- *Cantidad:* Varios.
- *Costo Estimado (MXN):* \$300.
- *Fuente de Financiamiento:* Recursos Propios.
- *Recurso:* Material de impresión y papelería (para documentación, artículos).
- *Cantidad:* Según necesidad.
- *Costo Estimado (MXN):* \$200.
- *Fuente de Financiamiento:* Recursos Propios.

■ **Imprevistos:**

- *Descripción:* Reserva para gastos no contemplados (aproximadamente 10 % del subtotal de gastos directos).

- *Costo Estimado (MXN)*: A definir una vez se tengan costos más precisos.
- *Fuente de Financiamiento*: Recursos Propios.

Nota: Los costos son estimaciones preliminares. Las necesidades y costos pueden variar a medida de que avanza el proyecto.

4.3.4. Infraestructura Requerida

La infraestructura necesaria para el proyecto incluye:

■ Instalaciones Físicas:

- Laboratorio de Cómputo de UPIITA: Para acceso a software especializado y desarrollo general.
- Espacio de Trabajo Personal: Para desarrollo y estudio.
- (Opcional) Laboratorio de Sistemas digitales de UPIITA: Si se requiere para pruebas con un robot físico, construcción de interfaces, o mediciones energéticas.

■ Equipo Principal:

- Computadora Personal (PC personal).
- Tarjeta de Desarrollo FPGA: Puede ser la Digilent Nexys 3 de los laboratorios de electronica de UPIITA, Intesc amiba 2 propia, o la proporcionada por el asesor.
- Robot Manipulador Físico: Pueden ser modelos desde 4 hasta 6 grados de libertad disponibles en el mercado que se ajusten al presupuesto o el proporcionado por el asesor.
- Instrumentación Electrónica: Multímetro, osciloscopio, fuente de alimentación regulada.

■ Software Especializado:

- Entorno de Desarrollo Integrado (IDE) para FPGA: Xilinx Vivado Design Suite (se utilizará la versión WebPACK gratuita o la disponible bajo licencia académica en UPIITA).
- Lenguajes de Programación: Python (versión 3.8 o superior), VHDL o Verilog (según la preferencia para el diseño en FPGA).
- Frameworks de Deep Learning: TensorFlow (versión 2.x con TensorFlow Lite) y/o PyTorch (con su módulo de cuantización).
- Simulador Robótico: CoppeliaSim (V-REP) en su versión educativa (gratuita).
- Software de Productividad: Un editor de LaTeX como overleaf, un gestor de referencias bibliográficas, software para creación de diagramas.
- Sistema de Control de Versiones: Git, con un repositorio en GitHub o GitLab.

4.3.5. Planeación de Actividades (Cronograma General)

La gestión detallada del proyecto, incluyendo la asignación de tareas específicas y el seguimiento del progreso, se realizará mediante *Sprints* de duración definida (ej., 2 a 4 semanas), conforme a la metodología *Scrum* adoptada (ver Sección 1.2.1). Para fines de este protocolo, se presenta a continuación una planeación general de alto nivel, distribuyendo los Épicos del proyecto a lo largo de las 36 semanas estimadas (18 semanas para Trabajo Terminal 1 - TT1, y 18 semanas para Trabajo Terminal 2 - TT2).

Fase TT1 (Semanas 1-18): Enfoque en Investigación, Diseño y Desarrollos Preliminares

■ *Semanas 1-4:*

EP01 Configuración del Entorno de Desarrollo y Simulación Robótica (CoppeliaSim, Python, interfaz básica).

INV01 Investigación exhaustiva del estado del arte: QNNs, control robótico, FPGAs, metodologías de evaluación energética.

DOC1 Inicio de la documentación de avances.

■ *Semanas 5-8:*

EP03 Diseño conceptual de la arquitectura de Red Neuronal (FP12-NN) para control. Generación/adquisición de datos de entrenamiento.

EP02 Diseño conceptual del controlador PID y su adaptación para FPGA.

DOC1 Continuación de la documentación.

■ *Semanas 9-13:*

EP03 Entrenamiento y validación (simulación Python) del modelo FP12-NN.

EP02 Inicio del desarrollo HDL y simulación del controlador PID para FPGA.

EP05 Investigación detallada y selección de técnicas y herramientas de cuantización.

DOC1 Continuación de la documentación.

■ *Semanas 14-18:*

EP02 Finalización de la implementación y pruebas básicas del PID en FPGA (interactuando con robot simulado).

EP05 Aplicación preliminar de cuantización (PTQ) al modelo FP12-NN para generar un primer QNN (ej., INT6).

DOC1 Preparación y finalización del informe de avances TT1.

ENT1 Entrega y presentación de avances TT1.

Fase TT2 (Semanas 19-36): Enfoque en Implementación en FPGA, Experimentación y Análisis

■ *Semanas 19-22:*

EP04 Diseño HDL e implementación del controlador FP12-NN en FPGA. Simulación y pruebas.

EP05/EP06 Refinamiento de la cuantización (posible QAT) y diseño HDL de los controladores QNN (INT6, INT4).

DOC2 Inicio de la redacción de la tesis con toda la documentación previa.

■ *Semanas 23-26:*

EP06 Implementación y pruebas en FPGA de los controladores QNN.

EP07 Diseño e implementación del sistema de medición/estimación de consumo energético y uso de recursos en la FPGA.

DOC2 Avance en la tesis.

■ *Semanas 27-31:*

EP08 Diseño e implementación del sistema de medición del rendimiento del control (precisión, latencia) en la FPGA.

EP09 Desarrollo de la interfaz de orquestación de pruebas y recolección de datos (PC - FPGA).

EP10 Inicio de la ejecución de experimentos comparativos.

DOC2 Continuación de la tesis.

■ *Semanas 32-36:*

EP10 Finalización de los experimentos. Análisis estadístico e interpretación de todos los resultados comparativos.

DOC2 Redacción final de la tesis, incluyendo conclusiones y trabajo futuro.

ENT2 Entrega de la tesis y preparación para la presentación final.

Nota: Este cronograma es una guía general de alto nivel. La asignación específica de tareas a los Sprints y su duración se definirá dinámicamente durante el proyecto, facilitando la adaptación a los hallazgos y desafíos que surjan. INV01, DOC1, DOC2, ENT1 y ENT2 son hitos o actividades transversales.

Fuentes – Referencias

Referencias

- [1] R. Eigenmann and D. J. Lilja, “Von neumann computers,” January 1998, purdue University / University of Minnesota.
- [2] All About Circuits. (2022, March) Neural network quantization: What is it and how does it relate to tiny machine learning? Accessed: 2025-05-10. [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/neural-network-quantization-what-is-it-and-how-does-it-relate-to-tiny-machine-learning/>
- [3] Wattco. (2024, julio) Controlador pid explicado: tipos, usos y operaciones. Accessed: 2025-05-01. [Online]. Available: <https://www.wattco.com/es/2024/07/controlador-pid-explicado/>
- [4] P. N. H. (2024, October) Artificial neural networks (anns) and their role in machine learning. Accessed: 2025-05-12. [Online]. Available: <https://medium.com/@prasanNH/artificial-neural-networks-anns-and-their-role-in-machine-learning-e19a214e46ec>
- [5] Q. Jaffery. (2023, September) Ann vs dnn. Accessed: 2025-05-12. [Online]. Available: <https://www.linkedin.com/pulse/ann-vs-dnn-qasim-jaffery/>
- [6] F. C. P. Kevin M. Lynch, *Modern robotics*. Reino Unido: Cambridge University Press, 2017.
- [7] J. A. J. Lora, “Design and implementation of a neuromorphic processing platform for robotic control systems,” Tesis de Doctorado en Ciencias de la Computación, Centro de Investigación en Computación, Instituto Politécnico Nacional, Ciudad de México, diciembre 2024.
- [8] Y. Park, J. Lee, D. Sim, Y. Cho, and C. Park, “Designing spiking neural network-based reinforcement learning for 3d robotic arm applications,” *Electronics*, vol. 14, no. 3, p. 578, 2025, article number 578. Check year if accessing preprint vs final version.
- [9] J. Chen, J. Li, Y. Li, and X. Miao, “Multiply accumulate operations in memristor crossbar arrays for analog computing,” *Journal of Semiconductors*, vol. 42, no. 1, p. 013104, 2021.
- [10] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, “Pruning and quantization for deep neural network acceleration: A survey,” *Neurocomputing*, vol. 461, pp. 370–403, 2021.
- [11] C. Bartolozzi, G. Indiveri, and E. Donati, “Embodied neuromorphic intelligence,” *Nature Communications*, vol. 13, p. 1024, 2022.

- [12] A. de Vries-Gao, “Artificial intelligence: Supply chain constraints and energy implications,” *Joule*, vol. 9, pp. 1–4, may 2025, published April 10, 2025.
- [13] L. Zhang, X. Hu, X. Liao, T. Zhou, Y. Peng, and Q. Zhao, “FLQ: Design and implementation of hybrid multi-base full logarithmic quantization neural network acceleration architecture based on FPGA,” *Signal Processing: Image Communication*, vol. 134, p. 117270, 2025, available online 16 January 2025.
- [14] I. I. Arikpo, F. U. Ogban, and I. E. Eteng, “VON NEUMANN ARCHITECTURE AND MODERN COMPUTERS,” *GLOBAL JOURNAL OF MATHEMATICAL SCIENCES*, vol. 6, no. 2, pp. 97–103, 2007.
- [15] Álvaro Domingo Reguero, S. Martínez-Fernández, and R. Verdecchia, “Energy-efficient neural network training through runtime layer freezing, model quantization, and early stopping,” *Computer Standards & Interfaces*, vol. 92, p. 103906, 2025, published online 10 October 2024.
- [16] Y. Zhang, B. Yin, M. D. Gomony, H. Corporaal, C. Trinitis, and F. Corradi, “Hardware/software co-design optimization for training recurrent neural networks at the edge,” *Journal of Low Power Electronics and Applications*, vol. 15, no. 1, p. 15, 2025, article number 15. Check year if accessing preprint vs final version.
- [17] B. Jiang, J. Chen, and Y. Liu, “Single-shot pruning and quantization for hardware-friendly neural network acceleration,” *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106816, 2023.
- [18] Y. S. Lee, Y.-H. Gong, S. W. Chung, Y. Kim, and E. Lim, “Scale-CIM: Precision-scalable computing-in-memory for energy-efficient quantized neural networks,” *Journal of Systems Architecture*, vol. 134, p. 102787, 2023, available online 30 November 2022.
- [19] K. M. Lynch and F. C. Park, “Robot control,” in *Modern Robotics*, L. Miller and K. Roberts, Eds. Berlin: Cambridge University Press, 2017, pp. 434–437.
- [20] L. Zanatta, F. Barchi, S. Manoni, S. Tolu, A. Bartolini, and A. Acquaviva, “Exploring spiking neural networks for deep reinforcement learning in robotic tasks,” *Scientific Reports*, vol. 14, p. 30648, 2024, article number 30648. Published online: 28 December 2024. Check year if different in final print.
- [21] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, J. Hays, C. Young, C. Bartolozzi, A. Majumdar, S. G. Cardwell, M. Payvand, S. Buckley, S. Kulkarni, H. A. Gonzalez, G. Cauwenberghs, C. S. Thakur, A. Subramoney, and S. Furber, “Neuromorphic computing at scale,” *Nature*, vol. 637, pp. 801–812, 2025, published online: 22 January 2025. Check year if accessing preprint vs final version.
- [22] A. Mehonic and A. J. Kenyon, “Brain-inspired computing needs a master plan,” *Nature*, vol. 604, no. 7905, pp. 255–260, 2022.

- [23] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [24] D. Vlasov, A. Minnekhanov, R. Rybka, Y. Davydov, A. Sboev, A. Serenko, A. Ilyasov, and V. Demin, “Memristor-based spiking neural network with online reinforcement learning,” *Neural Networks*, vol. 166, pp. 512–523, 2023.
- [25] M. V. Mark W. Spong, Seth Hutchinson, *Robot Modeling and Control*. Estados Unido: Wiley, 2005.
- [26] Intel. (2021) Llevar la computación neuromórfica al siguiente nivel con loihi 2. Accessed: 2025-05-01. [Online]. Available: <https://www.intel.la/content/www/xl/es/research/neuromorphic-computing-loihi-2-technology-brief.html>
- [27] X. Li, H. Jiang, R. Zhang, F. Tian, S. Huang, and D. Xu, “Robustness-aware 2-bit quantization with real-time performance for neural network,” *Neurocomputing*, vol. 455, pp. 12–22, 2021.
- [28] N. M. Rezk, T. Nordström, D. Stathis, Z. Ul-Abdin, E. E. Aksoy, and A. Hemani, “MOHAQ: Multi-Objective Hardware-Aware quantization of recurrent neural networks,” *Journal of Systems Architecture*, vol. 133, p. 102778, dec 2022.
- [29] K. Schwaber and J. Sutherland, *The Scrum Guide™*, November 2020, the Definitive Guide to Scrum: The Rules of the Game. Available at <https://scrumguides.org>. [Online]. Available: <https://scrumguides.org>
- [30] E. L. Cano, J. M. García-Camús, J. Garzás, J. M. Moguerza, and N. N. Sánchez, “A scrum-based framework for new product development in the non-software industry,” *Journal of Engineering and Technology Management*, vol. 61, p. 101634, 2021. [Online]. Available: <https://doi.org/10.1016/j.jengtecman.2021.101634>
- [31] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Berlin, Heidelberg: Springer-Verlag, 2008.
- [32] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul, “On-line computational scheme for mechanical manipulators,” *IEEE Transactions on Automatic Control*, vol. 25, no. 3, pp. 468–474, 1980.
- [33] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [34] S. Macenski, T. Foote, B. Gerkey, M. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

- [35] Universal Robots A/S, *UR3e User Manual - Original Instructions (e-Series)*, Universal Robots A/S, Odense, Denmark, 2021, version 5.11.
- [36] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, 1st ed. New York: John Wiley & Sons, 2006.
- [37] A. Savitzky and M. J. E. Golay, “Smoothing and differentiation of data by simplified least squares procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.
- [38] S. Pugh, *Total Design: Integrated Methods for Successful Product Engineering*. Wokingham, England: Addison-Wesley, 1991.
- [39] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.
- [40] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

Glosario